

Zero Touch Provisioning

student **Reynders Gregory**

studiegebied **Handelswetenschappen,**
Bedrijfskunde en Toegepaste informatica

graduaat **stysteem- en netwerkbeheer**

campus **Brugge**

academiejaar **2024-2025**

Voorwoord

Mijn naam is Reynders Gregory, ik ben student Systeem- en Netwerkbeheer. Voordat ik aan deze opleiding begon, werkte ik als kok, maar op een bepaald moment besepte ik dat ik toe was aan een nieuwe uitdaging. Mijn interesse in IT was er altijd al, maar eerder was het voor mij geen haalbare optie. Toen zich de kans voordeed om een carrièreswitch te maken, greep ik deze met beide handen.

De keuze voor een graduaatsopleiding in plaats van een bachelor kwam voort uit de inschatting dat deze studie beter aansloot bij mijn leerstijl en capaciteiten. Het was dan ook al een aantal jaar geleden dat ik nog op de schoolbanken had gezeten. Gedurende deze opleiding heb ik mijn technische vaardigheden ontwikkeld en praktijkervaring opgedaan in netwerkinfrastructuur.

Deze graduaatsproef vormt het bewijs van mijn inzet en harde werk. Het is niet alleen een afsluiting van mijn studie, maar ook een bevestiging dat ik de opgedane kennis correct kan toepassen in een professionele omgeving.

Tot slot wil ik graag mijn dank betuigen aan het netwerkteam van het AZ Sint-Jan voor hun begeleiding en waardevolle inzichten. Ook wil ik mijn docenten bedanken voor hun ondersteuning en inspirerende lessen.

Maar vooral wil ik mijn vriendin bedanken voor haar geduld, aanmoediging en steun, zowel op moeilijke momenten als bij elk klein succes. Zonder haar had ik deze opleiding niet kunnen voltooien.

Inhoudstafel

Voorwoord	3
Figurenlijst	7
Inleiding	8
1 De werkplek & Werkplekieren	9
1.1 AZ Sint-Jan Brugge	9
1.2 Werkplekieren	10
1.3 HPE/Aruba networking	10
2 Wat maakt mijn project relevant?	11
2.1 Huidige situatie	11
2.2 Toekomstsituatie	11
2.3 Documentatie	12
3 Informatie over de gebruikte producten	13
3.1 Hardware	13
3.1.1 Switch (Aruba 2930F & Aruba CX 6300M)	13
3.1.2 Mini-Computer (Dell Optiplex)	13
3.1.3 Ethernet kabel (UTP/FTP)	14
3.1.4 Cisco catalyst switch 2960	14
3.1.5 Usb ethernet adapter	15
3.2 Software	16
3.2.1 Windows Server 2022	16
3.2.2 Infoblox	16
3.2.3 Python	17
3.2.4 Netmiko	17
3.3 Netwerkkoncepten	18
3.3.1 IPv4 Adressen	18
3.3.2 Subnet + Masker	18
3.3.3 DHCP	18
3.3.4 SSH-protocol	19
3.3.5 TFTP	19
3.3.6 VLAN	19
3.3.7 VSF-Stack	19
4 Start van Zero Touch Provisioning met Aruba Os 2930F	21
4.1 Opstart	21
4.1.1 Onderzoek en opstart van de switch	21

4.2	Opbouw van de omgeving	23
4.2.1	Schema opzet	23
4.3	Windows server 2022.....	24
4.3.1	DHCP-Rol & Subopties.....	24
4.4	TFTP-Server Instellingen.....	28
4.5	Testen en Troubleshooten	29
4.5.1	Configuratiebestand.....	30
5	Zero Touch Provisioning.....	31
5.1	Schema	32
5.2	Aruba CX 6300M.....	32
5.3	Configuratie bestand CX 6300M	32
5.3.1	Variabelen	33
5.4	Configuratie en firmware bestanden	34
5.4.1	Autostacking.....	35
5.5	Infoblox.....	37
5.5.1	Ondervonden problemen.....	38
6	Netmiko.....	39
6.1	Wat doet het python script.....	39
6.2	Uitleg script	41
6.2.1	Aanmelding	41
6.2.2	Inlezen vlan.txt	42
6.2.3	VLAN-input van de gebruiker	43
6.2.4	Registreren members & interfaces	44
6.2.5	Verwijderen van overtollige members.....	46
6.2.6	Configureren van de management interface.....	47
6.2.7	VLAN aanmaken	48
6.2.8	Expliciet blackhole navragen	49
6.2.9	interfaces in de Blackhole Vlan	50
6.2.10	Link Aggregation Group.....	51
6.2.11	Client track ip.....	52
6.2.12	Laatste commando's	53
	Conclusie	54
	Referentie lijst	56
	Bijlagen	57
	Handleiding Zero Touch Provisioning ArubaCX 6300M & Python Script	57

Figurenlijst

Figuur 1: AZ Sint-Jan - Brugge - 2022	9
Figuur 2: HPe Aruba Networking Logo (Hpe Aruba Networking)	10
Figuur 3: Aruba Switch 2930F (HPe Aruba Networking)	13
Figuur 4: Aruba Switch CX6300M (HPe Aruba Networking)	13
Figuur 5: Dell Optiplex 7050 Micro (Dell Technologies).....	13
Figuur 6: Utp Kabel twisted pair.....	14
Figuur 7: Cisco Catalyst Switch (Cisco)	14
Figuur 8: USB Ethernet Adapter (Startech)	15
Figuur 9: Logo Windows Server 2022 (Microsoft)	16
Figuur 10: Logo Infoblox.....	16
Figuur 11: Logo Python.....	17
Figuur 12: Schema opstelling	23
Figuur 13: Screenshot Role Manager	24
Figuur 14: Screenshot define vendor classes	25
Figuur 15: Screenshot vendor class input	26
Figuur 16: Screenshot sub opties	27
Figuur 17: Screenshot Sub opties selecteren	28
Figuur 18: Screenshot TFTP64	29
Figuur 19: Screenshot aanpassing vendor class.....	31
Figuur 20: Schema opstelling netwerk Aruba CX6300M	32
Figuur 21: Screenshot Firmware update fail	34
Figuur 22: Screenshot Firmware update zonder Cisco Switch.....	35
Figuur 23: Foto voorbeeld Stacking interfaces (Hpe Aruba Networking)	36
Figuur 24: Voorbeeld VSF stacking interfaces.....	36
Figuur 25: aangepast voorbeeld interfaces VSF stacking voor Autostacking	36
Figuur 26: Screenshot Infoblox voorbeeld	37
Figuur 27 Schema werking python script.....	40
Figuur 28: Screenshot Script aanmeld gegevens	41
Figuur 29: Screenshot Script inlezen vlan.txt	42
Figuur 30: Screenshot Script input van de user	43
Figuur 31: Screenshot Script stacking & interfaces.....	44
Figuur 32: Screenshot Script verwijderen VSF members	46
Figuur 33 Screenshot Script configureren Mgmnt Interface	47
Figuur 34: Screenshot Script aanmaken VLAN	48
Figuur 35: Screenshot Script expliciet blackhole navragen	49
Figuur 36: Screenshot Script interfaces in blackhole VLAN	50
Figuur 37: Screenshot Script LAG	51
Figuur 38: Screenshot Script client track ip.....	52
Figuur 39: Screenshot Script laatste commando's.....	53

Inleiding

In moderne organisaties, en met name in ziekenhuizen, speelt een goed functionerend netwerk een essentiële rol. Elk apparaat, van bewakingscamera's en patiëntmonitors tot geavanceerde medische robots, is verbonden met het netwerk en afhankelijk van een snelle, veilige en betrouwbare gegevensoverdracht. Zonder een goed onderhouden netwerk kunnen kritieke processen vertraging oplopen of zelfs uitvallen, wat de zorgkwaliteit in gevaar brengt.

Het belang van een stabiel en veilig netwerk in de maatschappij kan niet worden onderschat. In de zorgsector is efficiënte data-uitwisseling cruciaal voor zowel de werking van medische apparatuur als de communicatie tussen zorgverleners. Daarom moet netwerkkapparatuur niet alleen up-to-date zijn met de nieuwste beveiligingspatches en software-upgrades, maar ook snel en efficiënt geïmplementeerd kunnen worden om te voldoen aan de steeds groeiende digitale en veiligheidseisen.

Tegelijkertijd staan veel organisaties, waaronder zorginstellingen, onder toenemende druk door besparingen op personeel, waardoor de werkdruk voor IT-teams blijft stijgen. Dit maakt het des te belangrijker om te investeren in robuuste, schaalbare en eenvoudig te beheren netwerkoplossingen die niet alleen de continuïteit waarborgen, maar ook bijdragen aan een efficiëntere werkomgeving.

Mijn graduaatsproef richt zich op de automatisering van dit proces met behulp van Zero Touch Provisioning (ZTP) voor Aruba-switches. Dit systeem stelt netwerkbeheerders in staat om nieuwe switches direct uit de verpakking aan te sluiten, waarna deze automatisch worden gedetecteerd en geconfigureerd via het netwerk.

Na deze initiële configuratie biedt een zelfontwikkeld Python-programma extra flexibiliteit. Hiermee kunnen netwerkbeheerders eenvoudig specifieke variabelen instellen per apparaat. Waar normaal gesproken elke switch afzonderlijk geconfigureerd moet worden, maakt dit systeem het mogelijk om tot maximaal 48 switches tegelijkertijd te configureren. Dit bespaart tijd, minimaliseert menselijke fouten en zorgt voor een efficiënte en kostdrukkende implementatie van netwerkkapparatuur.

In deze graduaatsproef wordt dieper ingegaan op de technische werking van Zero Touch Provisioning, de implementatie ervan binnen het AZ Sint-Jan, het zelfontwikkelde script, en de mogelijkheid om deze aanpak uit te rollen bij andere organisaties en met verschillende netwerk vendors .

1 De werkplek & Werkpleklerin

1.1 AZ Sint-Jan Brugge

Het Sint-Janshospitaal in Brugge heeft een lange ziekenhuistraditie die teruggaat tot 1150, toen de eerste hospitaalzaal werd gebouwd. Door de eeuwen heen werd het ziekenhuis uitgebreid en aangepast aan de veranderende noden van de gezondheidszorg. In 1796 kwam het onder het beheer van de Commissie van Burgerlijke Godshuizen, die later evolueerde naar de Commissie van Openbare Onderstand (COO) en uiteindelijk het Openbaar Centrum voor Maatschappelijk Welzijn (OCMW) werd in 1977.

Na de Tweede Wereldoorlog bleek de infrastructuur van het Sint-Janshospitaal verouderd en werd besloten een nieuw ziekenhuis te bouwen aan de rand van de stad. De bouw begon in 1966 en het nieuwe ziekenhuis werd officieel geopend op 18 maart 1977.

Het Sint-Franciscus Xaveriusziekenhuis (SFX) ontstond in 1927 uit de thuiszorgtraditie van de Zwartzusters, die sinds 1361 actief waren in Brugge. Onder leiding van Zuster Constance Foulon werd een deel van het klooster omgevormd tot een kliniek. Door de jaren heen groeide het ziekenhuis en onderging het meerdere renovaties en uitbreidingen, met belangrijke verbouwingen tussen 1980 en 2000.

Op 1 januari 2000 fuseerden Campus Sint-Jan en Campus Sint-Franciscus Xaverius tot het AZ Sint-Jan AV. Sindsdien streeft het ziekenhuis naar hoogwaardige, humane en betaalbare zorg, waarbij samenwerking met andere zorginstellingen en de focus op transmurale zorg centraal staan. (Ziekenhuis, sd)



Figuur 1: AZ Sint-Jan - Brugge - 2022

1.2 Werkpleklerin

In 2024 begon mijn werkpleklerin binnen het netwerkteam van het ziekenhuis. Dit team bestaat uit vijf leden, waaronder het hoofd van het netwerkteam en mijn mentor, Peter Van Rietvelde. Binnen het team wordt gewerkt met netwerkapparatuur. Dit zijn switches, access points en software om de IT-infrastructuur optimaal te laten functioneren. Het netwerk vormt een cruciale schakel in de dagelijkse werking van het ziekenhuis, aangezien het ondersteuning biedt aan medische apparatuur, patiëntendossiers, communicatie tussen afdelingen en tal van andere operationele processen.

Het huidige netwerk is grotendeels gebaseerd op Aruba-apparatuur, maar een aanzienlijk aantal switches en access points is inmiddels verouderd. Daarnaast zijn er nog steeds oude HPE Comware-switches in gebruik, die geleidelijk worden uitgefaseerd en vervangen door Aruba-switches. Dit is een langdurig en kostelijk proces, maar essentieel om de snelheid, stabiliteit en veiligheid te garanderen die moderne netwerken vereisen.

Tijdens mijn werkpleklerin heb ik nauw samengewerkt met specialisten uit verschillende netwerkdisciplines. Hierdoor heb ik niet alleen veel geleerd over netwerken in het algemeen, maar ook over de Aruba-hardware die in het ziekenhuis wordt gebruikt. Deze praktijkervaring was van grote waarde voor mijn verdere toekomst.

1.3 HPE/Aruba networking

Aruba Networks werd in 2002 opgericht door Keerti Melkote en Pankaj Manglik. De naam "Aruba" werd bewust gekozen om een gevoel van rust en betrouwbaarheid uit te stralen naar netwerkbeheerders die met hun producten werken. Samen met hun mentor, Dominic Orr, begonnen zij aan de ontwikkeling van een toekomst waarin WI-FI een essentiële technologie zou worden.

Door de jaren heen introduceerde Aruba een aantal baanbrekende producten, zoals AirWave, Aruba Instant en ClearPass, die een cruciale rol speelden in de opkomst van de BYOD (Bring Your Own Device) trend. In 2015 brak een nieuw hoofdstuk aan voor het bedrijf toen het werd overgenomen door Hewlett-Packard. Ondanks de overname werd beloofd dat Aruba als zelfstandig netwerkmerk binnen HP zou blijven opereren, waardoor het zijn identiteit en innovatieve kracht kon behouden (Melkote, 2021).



Figuur 2: HPE Aruba Networking Logo (HPE Aruba Networking)

2 Wat maakt mijn project relevant?

2.1 Huidige situatie

Netwerkproblemen komen terecht bij mijn mentor, Peter Van Rietvelde. Hij analyseert het probleem en maakt de beslissing over de volgende stap.

Wanneer een switch aan vervanging toe is, maakt hij een inschatting van wat nodig is om de betreffende datakast opnieuw op te bouwen. Daarbij houdt hij ook rekening met toekomstige noden, zoals compatibiliteit met nieuwe medische apparatuur. In sommige gevallen staan er momenteel twee of drie verouderde HPE Comware-switches opgesteld. Afhankelijk van de verwachte capaciteit en toekomstige uitbreidingen beslist hij of deze vervangen worden door drie of zelfs vier nieuwe switches.

Elke Comware-switch wordt waar mogelijk vervangen door het model van Aruba, de CX 6300M. In uitzonderlijke gevallen blijft de bestaande Comware nog wat langer staan.

Na deze beslissing start Peter met de configuratie van de nieuwe switches. Voor hem, als ervaren netwerkexpert, neemt het updaten van de firmware en het configureren van een standalone switch ongeveer anderhalf uur in beslag. Een stack van drie of vier switches vraagt ongeveer evenveel tijd als een standalone, al komen daar wel extra configuratiestappen bij kijken. Voor minder ervaren teamleden neemt dit proces meer tijd in beslag. Daarom worden er interne handleidingen of templates gebruikt als leidraad.

2.2 Toekomstsituatie

Met mijn project kan de tijd die nodig is om een switch gebruiksklaar te maken gehalveerd worden of zelfs meer.

Het doel is om een nieuwe switch, rechtstreeks uit de doos, automatisch te kunnen voorzien van een basisconfiguratie via een deployment switch, waarmee hij via de managementinterface verbonden wordt.

Zodra de switch verbonden is, krijgt hij automatisch een IP adres toegewezen via DHCP. Vervolgens haalt hij via een TFTP-server de meest recente firmware op, gevolgd door een vooraf gedefinieerde basisconfiguratie, die wordt toegepast op de running-config van de switch. Dit vormt de eerste stap in het Zero Touch Provisioning-proces.

Omdat de uiteindelijke configuratie afhankelijk is van de specifieke toepassing van de switch, moeten er nadien nog variabele instellingen toegevoegd worden. Hiervoor wordt een script gebruikt dat de gebruiker een aantal vragen stelt over bijvoorbeeld VLAN's, poortinstellingen en stack-members. Op basis van deze input genereert het script de aangepaste configuratie en stuurt deze automatisch door naar de switch.

Na het doorlopen van deze stappen is de switch volledig geconfigureerd en klaar om geplaatst te worden in een datakast. Dit proces verhoogt de efficiëntie voor netwerkbeheerders aanzienlijk, aangezien het veel tijd bespaart. Hierdoor kunnen zij zich richten op andere belangrijke taken binnen het netwerkbeheer, wat niet alleen de productiviteit verhoogt maar ook kostenbesparend werkt voor het ziekenhuis.

2.3 Documentatie

Nadat ik een duidelijk beeld had van de veranderingen die mijn project zou kunnen brengen op het bestaande systeem en nadat ik een overleg heb gehad over de verwachtingen die mijn mentor had over dit project, ben ik gestart met het verzamelen van relevante documentatie.

In eerste instantie richtte ik mijn onderzoek op Zero Touch Provisioning (ZTP), specifiek in combinatie met netwerkapparatuur van Aruba. Mijn eerste stap was dan ook het raadplegen van de officiële website en technische handleidingen van Aruba. Hier vond ik een aantal basisrichtlijnen over hoe ZTP werkt binnen hun ecosysteem.

Helaas bleek al snel dat de beschikbare informatie zich vooral beperkte tot oplossingen binnen de eigen Aruba-software. Aangezien ik ZTP wilde toepassen met behulp van externe middelen zoals een Windows Server en TFTP, volstond deze documentatie niet.

Daarom besloot ik mijn zoekstrategie bij te sturen. In plaats van te zoeken op “Aruba Zero Touch Provisioning”, ben ik me gaan richten op bredere termen zoals “provisioning with DHCP and TFTP”. Deze bredere aanpak leverde extra bronnen en inzichten op die mij hielpen om stap voor stap naar een werkbare eindoplossing toe te werken.

Een groot deel van de informatie die ik gevonden heb, was niet rechtstreeks van toepassing op mijn graduaatsproef, maar hielp me wel om bepaalde problemen op te lossen en om probleemoplossend te denken. Deze informatie vond ik voornamelijk via Reddit.

Voor bepaalde problemen die verband hielden met specifieke onderdelen van mijn graduaatsproef, was het vaak eenvoudiger om gerichte oplossingen te vinden dan telkens te zoeken op algemene termen zoals “Zero Touch Provisioning werkt niet”.

3 Informatie over de gebruikte producten

3.1 Hardware

3.1.1 Switch (Aruba 2930F & Aruba CX 6300M)

Een switch is een netwerkkapparaat dat ervoor zorgt dat alle bekabelde toestellen binnen een lokaal netwerk (LAN) met elkaar kunnen communiceren. Het verbindt onder andere computers, printers, servers, access points en andere netwerkkapparatuur met elkaar.



Figuur 3: Aruba Switch 2930F (HPE Aruba Networking)



Figuur 4: Aruba Switch CX6300M (HPE Aruba Networking)

3.1.2 Mini-Computer (Dell Optiplex)

Dit is de Dell OptiPlex 7050 Micro is een compacte zakelijke desktop-pc die nauwelijks groter is dan een boek. Het bevat een Intel Core i5 7500 (Kaby Lake genaamd), uitgebracht in 2016-2017 en ook 8 GB DDR4 RAM, en 256GB SSD-opslag.

Ook beschikt het over USB3.1 en een netwerkkaart die tot 1Gb aan snelheid ondersteunt.

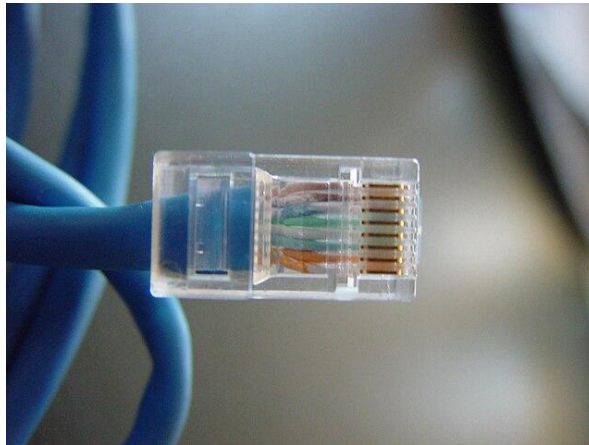


Figuur 5: Dell Optiplex 7050 Micro (Dell Technologies)

3.1.3 Ethernet kabel (UTP/FTP)

Een ethernetkabel (netwerkkabel) is een kabel die aan beide uiteinden is voorzien van een RJ-45 connector en wordt gebruikt om apparaten zoals computers, printers of access points met een switch of router te verbinden binnen een netwerk.

In de kabel bevinden zich acht koperdraden, gegroepeerd in vier getwiste paren, elk met een eigen kleurcodering. Deze draden zorgen voor de daadwerkelijke overdracht van netwerkverkeer.



Figuur 6: Utp Kabel twisted pair

3.1.4 Cisco catalyst switch 2960

Deze Cisco-switch heeft hetzelfde doel als de beide Aruba-switches, namelijk het doorsturen van netwerkverkeer (switchen) tussen aangesloten apparaten. Het gaat hier wel om een ouder model, waardoor de prestaties niet meer te vergelijken zijn met de recentere Aruba-modellen.

De meeste poorten op deze switch ondersteunen slechts een maximale snelheid van 100 Mbit/s.



Figuur 7: Cisco Catalyst Switch (Cisco)

3.1.5 Usb ethernet adapter

Een netwerkadapter is een component dat een apparaat in staat stelt om verbinding te maken met een bekabeld netwerk (via ethernet) of een draadloos netwerk (WI-FI). In de meeste gevallen is deze adapter ingebouwd in computers, laptops of andere toestellen, maar er bestaan ook externe netwerkadapters die apart kunnen worden aangekocht en eenvoudig via bijvoorbeeld een USB-poort kunnen worden aangesloten.



Figuur 8: USB Ethernet Adapter (Startech)

3.2 Software

3.2.1 Windows Server 2022

Voor dit project maak ik gebruik van Windows Server 2022. Dit besturingssysteem biedt de mogelijkheid om op een snelle en gebruiksvriendelijke manier via een GUI (Graphical User Interface) een DHCP-server in te stellen, wat noodzakelijk is voor het Zero Touch proces. Windows Server is een serverbesturingssysteem ontwikkeld door Microsoft en wordt veel gebruikt in bedrijfsomgevingen om verschillende netwerkdiensten en serverrollen te beheren. Voor dit project was vooral de DHCP-rol belangrijk (Scholten, 2024).



Figuur 9: Logo Windows Server 2022 (Microsoft)

3.2.2 Infoblox

Infoblox is een platform dat binnen het ziekenhuis wordt gebruikt voor DHCP, DNS en TFTP. Dit platform biedt organisaties van bedrijven en overheden tot serviceproviders wereldwijd geavanceerde oplossingen op het gebied van DNS, DHCP en IP-adresbeheer (IPAM). Infoblox zorgt dat netwerken vanaf de basis te beveiligen en centraal te beheren zijn. Ook beschermt het de organisaties tegen een steeds groter wordend aantal cyberdreigingen, zoals malware en gerichte aanvallen op het netwerk. Door middel van centraal beheer kunnen complexe infrastructuren eenvoudiger worden beheerd (Truegen, sd).



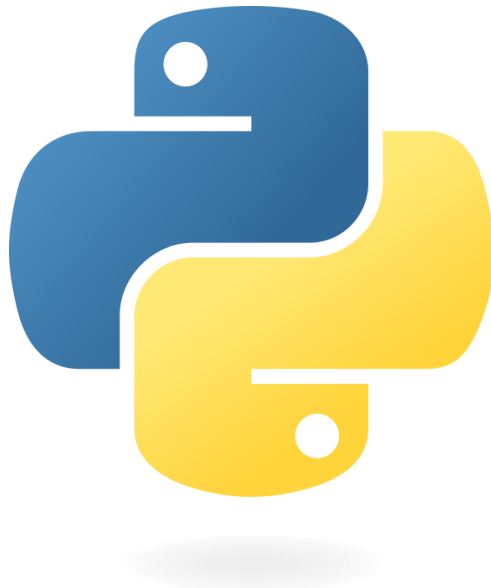
Figuur 10: Logo Infoblox

3.2.3 Python

Python is een programmeertaal die eenvoudig en leesbaar is. Dankzij de duidelijke structuur en lage learning curve is Python erg populair bij beginnende programmeurs, maar ook ervaren ontwikkelaars kiezen voor Python. Een kenmerk van Python is dat inspringing (indentatie) wordt verplicht is voor de correcte werking van de code.

Python is ook platformonafhankelijk: het werkt op bijna elk besturingssysteem en is bovendien gratis, wat het zeer toegankelijk maakt voor iedereen.

Python is ook veelzijdig en wordt gebruikt voor webontwikkeling, data-analyse, automatisering en scripting. Hiervan is de library Netmiko een goed voorbeeld (Workingtalent, sd).



Figuur 11: Logo Python

3.2.4 Netmiko

Netmiko is een bibliotheek van Python, die het werk met routers, switches en firewalls eenvoudiger maakt via het SSH-Protocol. Door middel van het opmaken van een pythonscript met de configuraties, kun je hiermee door één druk op de knop meerdere configuraties uitvoeren of data van het netwerkapparatuur afhalen(bijv. de running-configuratie). Netmiko is een waardevol hulpmiddel binnen in het netwerkbeheer (Pynetlabs, 2024).

3.3 Netwerkoncepten

3.3.1 IPv4 Adressen

Een IP-adres is een uniek adres dat aan elk apparaat binnen een netwerk wordt toegekend. Het zorgt ervoor dat apparaten met elkaar kunnen communiceren binnen dat netwerk. Een IP-adres bestaat uit twee delen: het **netwerkdeel**, dat aangeeft tot welk netwerk het apparaat behoort, en het **hostdeel**, dat het specifieke apparaat binnen dat netwerk identificeert. Dit zijn private IP-adressen.

Er zijn ook IP-adressen die specifiek worden gebruikt voor communicatie via het internet. Dit zijn de publieke IP-adressen, die wereldwijd uniek zijn en door internetproviders worden toegekend (Kapersky, sd).

3.3.2 Subnet + Masker

Een subnet is een logisch onderdeel van een groter netwerk, waarbij dat netwerk wordt opgedeeld in kleinere, beter beheersbare segmenten. Deze segmentatie gebeurt aan de hand van een subnetmasker, dat bepaalt welk deel van het IP-adres het netwerk-ID vormt en welk deel het host-ID.

Dankzij deze opdeling kunnen routers of Layer 3-switches bepalen waar netwerkverkeer naartoe moet, doordat ze weten of een bestemming zich binnen hetzelfde subnet bevindt of buiten het huidige netwerksegment (Providercheck, 2022).

3.3.3 DHCP

Dynamic Host Configuration Protocol (DHCP) is een netwerkprotocol waarmee een computer of ander netwerkapparaat automatisch netwerkinstellingen kan krijgen van een DHCP-server. Dit betekent dat het apparaat automatisch een IPv4-adres toegewezen krijgt.

Naast het IP-adres kan een DHCP-server ook andere informatie meesturen, zoals het subnetmask, de gateway, DNS-servers, en andere DHCP-subopties. Deze extra instellingen worden automatisch toegepast op elk apparaat dat verbinding maakt met het netwerk (Wikipedia, sd).

3.3.4 SSH-protocol

Secure Shell is een protocol dat zich bevindt op de toepassingslaag van het OSI-model. SSH maakt veilige, versleutelde communicatie mogelijk tussen verschillende netwerkapparaten, waaronder computers, servers en switches en dient als vervanging voor oudere, minder veilige protocollen zoals Telnet.

Met SSH kan je op afstand inloggen op een ander systeem en commando's uitvoeren via een command line interface. Alle gegevens, zoals gebruikersnamen, wachtwoorden en opdrachten, worden versleuteld verzonden, waardoor ze niet leesbaar zijn voor mogelijke afliuisteraars.

Een belangrijk voordeel van SSH is de ondersteuning voor public/private key-authenticatie. Hiermee kunnen gebruikers zich veilig aanmelden zonder wachtwoord, wat handig is bij geautomatiseerde scripts of toepassingen waarbij interactie niet gewenst is. Daarnaast is deze methode ook veiliger dan traditionele wachtwoorden, omdat de privésleutel lokaal wordt bewaard en niet over het netwerk wordt verzonden (Mensink, 2022).

3.3.5 TFTP

Trivial File Transfer Protocol is een simpel protocol om bestanden over een netwerk over te versturen. TFTP gebruikt geen authenticatie en encryptie, dit betekent dat data in leesbare tekst wordt verstuurd. Dit maakt het niet zo veilig als het over onbeveiligde of openbare netwerken wordt gebruikt, omdat de gegevens makkelijk onderschept en gelezen kunnen worden.

TFTP wordt vooral gebruikt bij kleine bestanden binnen een lokaal netwerk (one.com, sd).

3.3.6 VLAN

Virtual Local Area Network is een virtueel netwerk dat een groep apparaten groepeerd binnen een fysiek netwerk. Ook al zijn de apparaten fysiek verspreid, vormen ze via een VLAN één logisch netwerksegment.

3.3.7 VSF-Stack

Met VSF (Virtual Switching Framework) kun je twee of meer Aruba-switches met elkaar koppelen via stacking-poorten, waardoor ze samen functioneren als één logisch geheel. Je kunt hierbij tot maximaal 10 switches combineren, afhankelijk van het switchmodel.

De fysieke koppeling tussen de switches gebeurt met behulp van een DAC-kabel (Direct Attach Copper), die je aansluit op twee zelf te kiezen poorten, zoals bijvoorbeeld poort 49 en 50.

Zodra de stack is opgezet, gedragen de switches zich als één virtuele switch. Dit betekent dat je slechts één keer een configuratie hoeft uit te voeren voor de hele stack. Ook wordt er slechts één management-IP-adres toegekend aan de volledige stack, dit maakt het beheren ervan eenvoudiger.

4 Start van Zero Touch Provisioning met Aruba Os 2930F

Aan het begin van dit project had ik slechts een beperkt beeld van hoe het volledige proces zou verlopen. Er was nog geen uitgewerkt plan, wat me ertoe aanzette om eerst grondig onderzoek te doen. Door relevante documentatie en technische informatie te raadplegen, kreeg ik meer inzicht in de manier waarop ik het project het best kon aanpakken en waar ik concreet moest starten.

Aangezien het idee voor dit project afkomstig was van mijn stagementor en bedoeld is voor implementatie binnen het ziekenhuis, was het vanzelfsprekend om te werken met Aruba-switches. Het volledige netwerk in het ziekenhuis is immers opgebouwd uit Aruba-netwerkapparatuur, wat deze keuze logisch en praktisch maakte.

In het volgende hoofdstuk wordt de technische opbouw van het Zero Touch Provisioning-proces toegelicht. Daarbij komen onder andere de configuratie van de gebruikte hardware, het gedrag van de Aruba-switches tijdens het provisioning proces en de verdere technische stappen in detail aan bod.

Aangezien er op het moment dat ik aan dit project wilde beginnen nog geen Aruba CX 6300M beschikbaar was, ben ik gestart met een ArubaOS 2930F-switch. Op die manier kon ik de werking van Zero Touch Provisioning verkennen en wat inzicht opdoen over het gedrag en de configuratie van het systeem.

4.1 Opstart

4.1.1 Onderzoek en opstart van de switch

De opstartfase van mijn graduaatsproef verliep niet zonder moeilijkheden. In eerste instantie was het belangrijk om een goed inzicht te krijgen in wat Zero Touch Provisioning (ZTP) precies is en hoe het technisch werkt. Ik richtte me daarbij vooral op het moment waarop een switch tijdens het opstarten automatisch zijn ZTP-proces activeert, en welke stappen dit proces precies nodig heeft.

Wanneer de switch in gebruik wordt genomen en aan de elektriciteit komt te liggen, doorloopt het eerst een aantal stappen om te testen of zijn hardware in orde is en klaar om te gebruiken.

4.1.1.1. Stap 1: power on self test

Als de switch wordt ingeschakeld, gaat de Power-LED aan en start de switch met een Power-On Self Test. Tijdens deze fase controleert het toestel of alle hardware in het chassis correct werkt. Dit omvat onder andere de CPU, het werkgeheugen (RAM), het flashgeheugen en de netwerkinterfaces.

In deze fase worden ook de ventilatoren getest, en de interne sensoren die informatie verzamelen over de temperatuur en stroomtoevoer. Deze controles zijn belangrijk om te garanderen dat het toestel veilig en stabiel kan opstarten.

4.1.1.2 Stap 2: laden van besturing systeem

Na de Power-On Self Test succesvol is doorlopen en er geen fouten of afwijkingen zijn ontdekt, begint de switch met het laden van het besturingssysteem (Operating System) vanuit het interne flashgeheugen. In de meeste gevallen zijn hierin twee firmware-images aanwezig: een Primary en een Secondary.

Mocht er in de toekomst een probleem optreden tijdens het opstarten, bijvoorbeeld door een corrupte firmware, dan biedt de switch tijdens het bootproces de mogelijkheid om handmatig te kiezen tussen het primary of secondary firmwarebestand. Dit is bijzonder handig voor herstel- of onderhoudssituaties, waarbij een alternatieve versie van het besturingssysteem nodig is.

4.1.1.3 Stap 3: configuratiebestand lokaliseren

Zodra het besturingssysteem volledig is opgestart, controleert de switch of er een configuratiebestand aanwezig is in het flashgeheugen. Als er geen configuratiebestand wordt gevonden, start de switch op in de factory default-modus, met standaardinstellingen. Als er wel een configuratiebestand beschikbaar is, wordt dit automatisch geladen in de running-config van de switch, zodat het toestel direct klaar is voor gebruik met de vooraf ingestelde instellingen.

4.1.1.4 Stap 4: Activeren van DHCP-Client

Op dit punt wordt de DHCP-client van de switch geactiveerd. Dit betekent dat de switch via het netwerk op zoek gaat naar een DHCP-server om automatisch een IPv4-adres te

verkrijgen. Het toegekende adres wordt toegewezen aan de management-VLAN, die standaard is ingesteld op VLAN 1. Dit is het punt dat belangrijk is voor mijn opstelling.

4.1.1. 5 Stap 5: Klaar om te gebruiken

De switch is nu klaar voor gebruik. De consolepoort wordt geactiveerd, waarna aanmelden mogelijk is via een eerder geconfigureerde gebruiker als de switch opstart in de standaardconfiguratie, is er geen gebruikersnaam of wachtwoord vereist. Dit kan daarna via de password manager worden ingesteld.

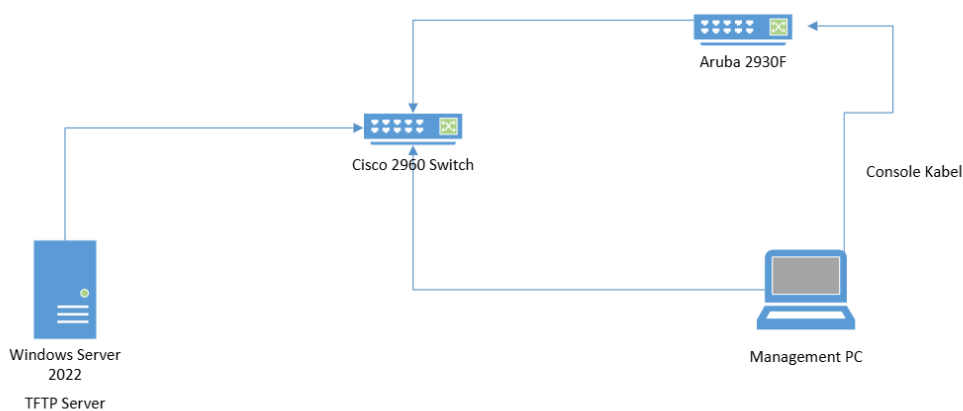
4.2 Opbouw van de omgeving

Om alle toestellen met elkaar te verbinden, heb ik gebruikgemaakt van een Cisco 2960-switch. Ook al is dit een ouder model , was hij nog perfect voor het doel van deze opstelling.

In deze setup heb ik de mini-pc, de ArubaOS 2930F-switch en mijn eigen computer aangesloten op de Cisco-switch, zodat alle apparaten met elkaar verbonden waren binnen hetzelfde netwerk.

Ook heb ik mijn computer via een consolekabel verbonden met de Aruba-switch. Op die manier kon ik via de seriële verbinding in real time volgen wat er precies gebeurde (of niet gebeurde) tijdens het opstartproces en de configuratie van de switch.

4.2.1 Schema opzet



Figuur 12: Schema opstelling

4.3 Windows server 2022

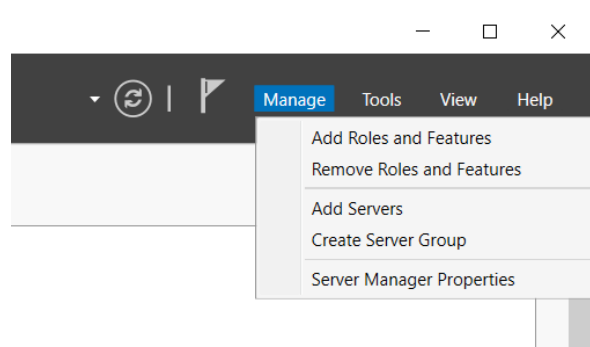
Het belangrijkste onderdeel van een Zero Touch opstelling is dat een switch automatisch geconfigureerd kan worden via het netwerk, met weinig tot geen tussenkomst van een netwerkbeheerder. Het is dan ook belangrijk dat de switch bij het opstarten via DHCP een IPv4-adres toegewezen krijgt, zodat hij contact kan maken met andere netwerkdiensten zoals een TFTP server.

Er zijn verschillende manieren om een DHCP-server op te zetten. In mijn geval overwoog ik twee opties: een Windows Server-omgeving met de DHCP-rol, of een DHCP-server op basis van Ubuntu Linux. Uiteindelijk koos ik ervoor om Windows Server 2022 te installeren op een Dell mini-pc.

De belangrijkste reden voor deze keuze is het gebruiksgemak een Windows Server omgeving is makkelijk op te zetten dankzij de grafische gebruikersinterface (GUI), dit maakt het configureren van de DHCP ook makkelijker. Hoewel een Ubuntu lichter zou zijn in betrekking van systeemvereisten, was dat voor dit project niet belangrijk. Ik beschik al over ervaring met Windows Server, wat de keuze logisch maakte.

4.3.1 DHCP-Rol & Subopties

Vooraleer ik met de rest van de opstelling kon beginnen, moest ik er eerst voor zorgen dat mijn Windows Server een statisch IP-adres kreeg. Dit was belangrijk om ervoor te zorgen dat het toestel altijd op hetzelfde adres bereikbaar bleef. Daarna kon ik de DHCP-rol installeren op de Windows Server. Deze rol is standaard beschikbaar binnen Windows Server en kan eenvoudig toegevoegd worden via het Server Manager-dashboard. De installatie verloopt via een overzichtelijke wizard waarin je stap voor stap begeleid wordt.

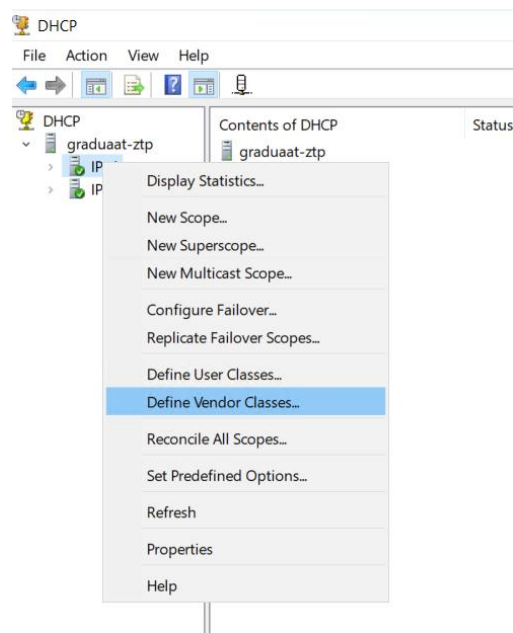


Figuur 13: Screenshot Role Manager

Na het succesvol toevoegen van de rol kan je via 'Tools' de DHCP-beheerconsole openen. Vanaf dat moment is de DHCP-rol operationeel en klaar voor verdere configuratie.

Hoewel deze installatie op zich weinig complexiteit met zich meebrengt, vormt ze de basis voor het verdere verloop van het project.

Het aanmaken van subopties begint bij het definiëren van een vendor class. Hiermee geef je een identificatie mee aan de DHCP-server die overeenkomt met het type of de versie van de switch. Op die manier kan de DHCP-server correct herkennen welk toestel een aanvraag doet en automatisch de juiste scope en bijbehorende opties toepassen. Deze vendor class wordt toegevoegd via het IPv4-menu in het DHCP venster, door met de rechtermuisknop te klikken en 'Define Vendor Classes' te selecteren.



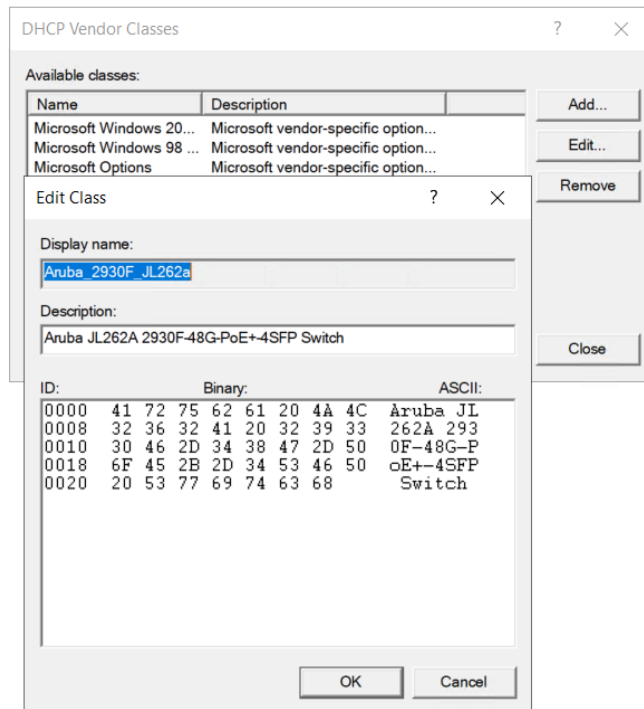
Figuur 14: Screenshot define vendor classes

Door op 'Add' te klikken, kun je een nieuwe vendor class toevoegen.

Display Name: Een herkenbare naam (bijvoorbeeld de switchnaam).

Description: Optioneel.

ASCII Field: Vul exact in wat je krijgt met het commando: **"show dhcp client vendor-specific"**



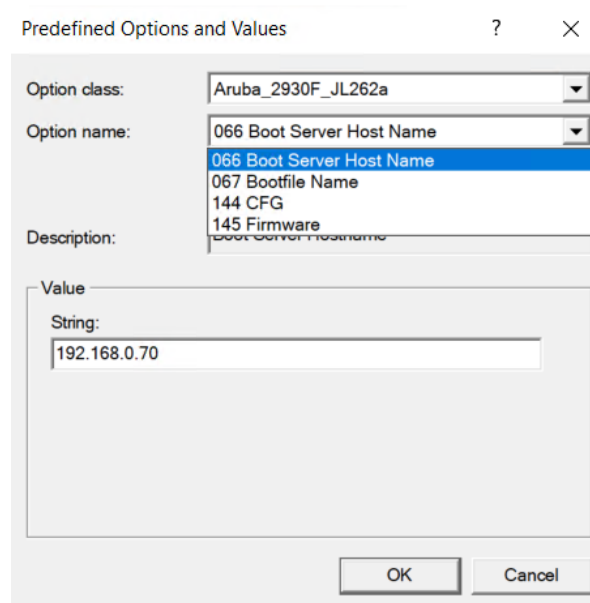
Figuur 15: Screenshot vendor class input

In het geval van Zero Touch Provisioning gaat het hier om het pad naar een configuratiebestand en eventueel een firmwarebestand. Deze informatie wordt meegegeven doormiddel van een aantal subopties.

Aangezien niet al deze opties standaard in Windows Server DHCP aanwezig zijn, moeten ze eerst toegevoegd worden via de predefined options.

De configuratie begint terug bij de DHCP en klik je met de rechtermuisknop op IPv4 en selecteer je 'Set Predefined Options'. Klik op 'add' en maak deze opties aan. De namen kunnen zelf gekozen worden, de value moet altijd op string staan. Al deze opties zijn belangrijk.

- 066 Boot Server Host name : Geeft het IP-adres of de hostnaam van de TFTP-server mee aan de client. Dit wordt als string ingevuld.
- 067 Bootfile name : Specificeert de naam van het bestand dat de switch moet ophalen bij de server die je opgeeft in optie 066.
- 144 CFG: Vendor-specifieke optie voor Aruba. Wordt gebruikt om een configuratiebestand of pad door te geven dat automatisch moet geladen worden bij de eerste opstart.
- 145 Firmware: Wordt gebruikt om een firmwarebestand mee te geven dat de switch moet downloaden en installeren.

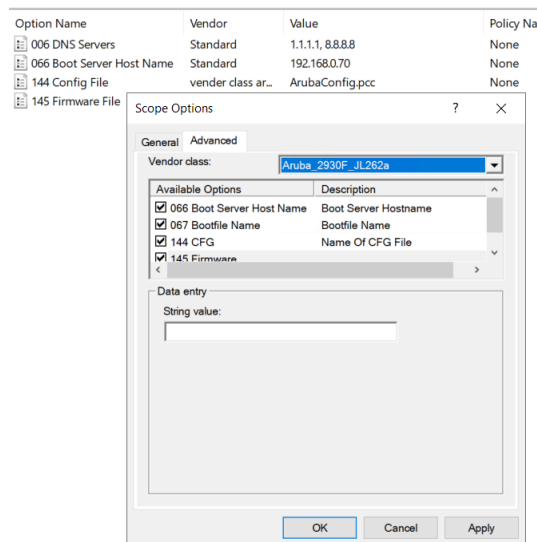


Figuur 16: Screenshot suboptions

Om de aangemaakte subopties te gebruiken, moeten ze worden gekoppeld aan een DHCP-scope. Na het aanmaken van deze scope kunnen de opties geconfigureerd worden via de scope options. Binnen de scope-instellingen is het mogelijk om via het tabblad 'Advanced' een vendor class te selecteren, zoals bijvoorbeeld Aruba_2930F_JL262A.

Nu deze vendor class is geselecteerd, verschijnen de vier zelf aangemaakte opties in de lijst. Door deze aan te vinken en de juiste stringwaarden in te vullen worden ze actief binnen de scope.

- 066 Boot server host name : het Ipv4 adres van de TFTP-server
- 067 Bootfile name : Naam van de configuratie file met extensie .pcc
- 144 configuratie (CFG) : Naam van de configuratie file met extensie .pcc
- 145 Firmware: exacte naam van de firmware file met extensie .swi



Figuur 17: Screenshot Sub opties selecteren

De DHCP-opties 066 en 067 zijn standaard beschikbaar in Windows Server en kunnen dus meteen gebruikt worden, zonder dat ze specifiek opnieuw aangemaakt moeten worden onder de vendor class van de Aruba 2930F. Toch heb ik ervoor gekozen om deze opties ook op te nemen binnen de aangemaakte vendor class, om het geheel overzichtelijker te houden en alle ZTP-gerelateerde instellingen gegroepeerd te beheren.

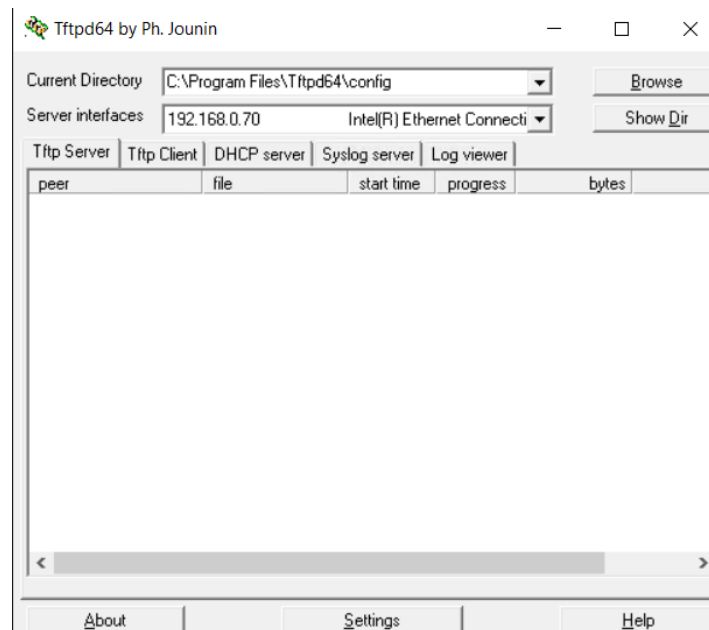
4.4 TFTP-Server Instellingen

Als TFTP-Server heb ik ervoor gekozen om gebruik te maken van de open-source tool TFTP64. Deze applicatie is licht en makkelijk te gebruiken.

Na het downloaden en installeren heb ik als eerste stap in de rootmap van de TFTP-server een aparte map aangemaakt met de naam "config", om alle benodigde bestanden overzichtelijk te groeperen. Vervolgens heb ik deze map geselecteerd als 'Current Directory' in de TFTP64-interface, zodat het programma weet waar het de configuratie en firmwarebestanden moet zoeken.

Hier stelde ik in het programma ook de serverinterface in. Ik selecteerde het statische IP-adres dat ik eerder op mijn Windows Server had aangegeven.

Ook biedt TFTP64 een log viewer, waarmee ik in realtime kan opvolgen of de switch verbinding maakt met de TFTP-server en of de bestanden worden aangevraagd en doorgestuurd.



Figuur 18: Screenshot TFTP64

4.5 Testen en Troubleshooten

Alles was klaar voor gebruik en alle toestellen waren verbonden met de Cisco-switch. Aangezien deze switch enkel dienst deed om netwerkverkeer door te sturen, was er geen specifieke configuratie nodig.

De eerste testfase begon met het starten van een Remote Desktop-sessie naar mijn Windows Server. Van hieruit kon ik controleren of er een DHCP-lease was toegekend aan de Aruba-switch. Ook maakte ik via een consolekabel verbinding met de switch om te controleren of ze effectief een IP-adres had ontvangen. Zodra dit bevestigd was, deed ik een ping-test van de server naar de switch en omgekeerd om na te gaan of de verbinding correct werkte.

Aangezien alles werkte naar behoren, besloot ik een eerste en eenvoudig configuratiebestand via TFTP te uploaden naar de switch. Eerst dacht ik dat ik simpelweg een tekstbestand met CLI-commando's kon maken en dat de switch deze automatisch zou lezen en uitvoeren. Dit was fout en achteraf bekeken was het logisch dat dit zo niet werkte. Om dit probleem op te lossen, gaf ik de switch eerst een minimale configuratie via de CLI. Daarna ging ik via de lokale web interface van de switch (Web GUI) om de running-config te downloaden. Het bestand werd opgeslagen met de extensie .pcc, wat geen standaard extensie is, maar dat maakt voor de switch zelf niet uit. De inhoud is belangrijker dan de extensie.

Na het downloaden heb ik het bestand manueel aangepast: ik voegde een hostname, een IP-adres en enkele VLAN's toe. Bij een nieuwe test kon ik in de log viewer van de TFTP-server zien dat het configuratiebestand effectief werd aangevraagd en verzonden.

Tijdens het opstartproces kreeg de switch een IP-adres toegewezen samen met de sub-opties, Waardoor het configuratiebestand gevonden werd. Ik ondervond wel nog enkele problemen met de syntaxis. De Aruba 2930F is redelijk gevoelig voor overbodige witruimtes, extra enters of fout geplaatste spaties. Deze fouten heb ik er dan uitgehaald en alles werkte perfect.

4.5.1 Configuratiebestand

Na mijn eerste succesvolle test met een zelfgemaakt configuratiebestand, kreeg ik via mijn stage de kans om een echte configuratie uit te werken voor een switch die vervangen moest worden.

Na het voltooien van deze configuratie heb ik de running-config van de switch gedownload. Ik zag snel dat bepaalde configuraties die ik had ingevoerd, niet zichtbaar waren in het gegenereerde configuratiebestand. Dat was belangrijk want de switch leest bij Zero Touch Provisioning uitsluitend de lijnen die effectief in de running-config staan.

Om zeker te zijn van een correct werkende configuratie, ben ik volledig opnieuw begonnen. Tijdens het configureren heb ik stap voor stap geëvalueerd welke lijnen wel en niet werden opgeslagen in de running-config. De commando's die ontbraken, heb ik apart genoteerd in een tekstbestand. Het ging slechts om een aantal lijnen, maar deze moesten dan handmatig nog toegevoegd worden.

Deze eerste testfase was een goede eerste stap, maar nu kon ik overschakelen naar de Aruba CX 6300M, de switch waarop mijn project werkelijk zal worden toegepast.

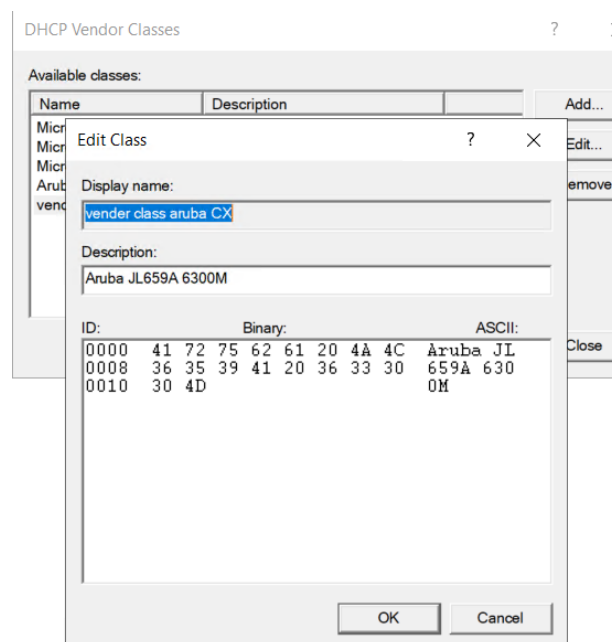
5 Zero Touch Provisioning

De eerste stappen van dit proces werden eerder al uitgevoerd met de Aruba OS 2930F. Hierdoor kon ik dezelfde basisopstelling hergebruiken voor de configuratie van de Aruba CX 6300M, met enkele kleine aanpassingen. In deze nieuwe opstelling liet ik de Cisco-switch achterwege aangezien deze tijdens een eerdere testfase voor problemen zorgde. Dit aspect wordt verder toegelicht in een volgend onderdeel.

Daarnaast moest enkel nog de Vendor Class Identifier aangepast worden aan het model van de Aruba CX 6300M.

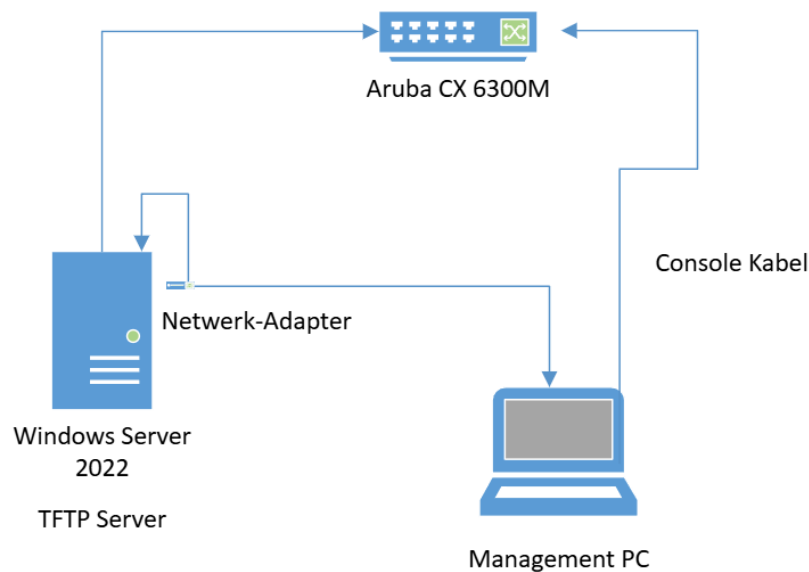
Bij nader inzien heb ik ervoor gekozen om een volledig nieuwe DHCP-range aan te maken, samen met een aparte vendor class en de bijhorende sub opties. Deze aanpak leek mij het meest praktisch, aangezien ik ervan uitging dat ik nog verdere tests zou moeten uitvoeren met de Aruba 2930F-switch.

Uiteindelijk bleek dit niet meer nodig te zijn. Toch heb ik deze configuratie behouden, voor het geval ik ze op een later moment toch nog nodig zou hebben.



Figuur 19: Screenshot aanpassing vendor class

5.1 Schema



Figuur 20: Schema opstelling netwerk Aruba CX6300M

5.2 Aruba CX 6300M

Op veel punten vertonen deze switches belangrijke gelijkenissen, waardoor ik opnieuw dezelfde werkwijze heb toegepast als bij de 2930F in fase 1. Ik ben gestart met het opstellen van een configuratie op een switch die voorbereid moest worden. Hierna controleerde ik of alle instellingen correct opgenomen werden in de running-configuratie. Daarna probeerde ik de configuratie via de web interface te downloaden.

Dit bleek geen goede keuze. De configuratie werd geëxporteerd als een .JSON-bestand, dat amper leesbaar en moeilijk aanpasbaar was. Als alternatief heb ik de uitvoer van het commando `show running-config` manueel gekopieerd en geplakt in een tekstbestand om verder te kunnen werken.

5.3 Configuratie bestand CX 6300M

Bij deze test werden alle ingevoerde configuraties correct opgenomen in de running-config en waren ze volledig zichtbaar. Aangezien deze configuratie afgestemd was op een concrete situatie, bleken een aantal instellingen niet universeel toepasbaar. Daarom heb ik de configuratie herleid tot een generieke basisconfiguratie waarin alle variabele gegevens werden verwijderd.

Dit was naar mijn mening, het belangrijkste onderdeel van het Zero Touch proces.

Nu beschikte ik over een herbruikbare template die als startpunt dient voor alle toekomstige configuraties.

5.3.1 Variabelen

Nadat de herbruikbare template afgewerkt was en succesvol getest werd, was het belangrijk om goed te documenteren welke onderdelen nog niet standaard in de basisconfiguratie of template konden worden opgenomen. Deze documentatie diende als leidraad om later te bepalen welke instellingen of aanpassingen nog manueel moesten worden toegevoegd.

- **Hostname:** Deze wordt aangepast op basis van de dienst die de switch in gebruik neemt. Elke switch krijgt dus een unieke naam, afhankelijk van de toepassing en locatie.
- **VLAN's:** Niet elke VLAN wordt standaard op elke switch geconfigureerd. Enkel de management VLAN wordt consequent toegepast op alle switches.
- **Blackhole VLAN:** Alle interfaces moeten toegewezen worden aan de blackhole VLAN.
- **Management IPv4-adres:** moet worden voorzien van een vast IPv4-adres.
- **Locatie:** De fysieke locatie van de switch wordt opgenomen in de configuratie met een duidelijke en consistente naamgeving.
- **Link Aggregation (LAG):** alle relevante VLAN's die op de switch actief zijn, moeten expliciet worden toegestaan op de LAG.
- **SNMP server location & description:** Deze velden krijgen de zelfde naam als de locatie naamgeving.
- **Client Track IP:** Voor deze functie moeten de juiste IP-adressen of instellingen nog worden toegevoegd aan de configuratie, afhankelijk van de gewenste trackingmethodiek.

5.4 Configuratie en firmware bestanden

Net zoals bij de Aruba 2930F wordt bij de Aruba CX via de DHCP-server een IPv4-adres gegeven, samen met enkele subopties. Deze opties geven de switch informatie over de locatie en namen van de configuratie en firmwarebestanden die hij moet ophalen.

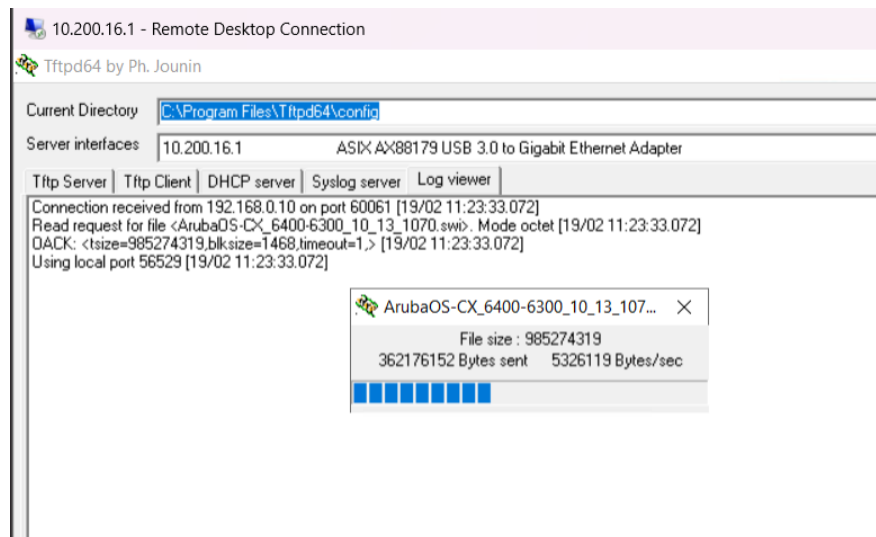
Tijdens de eerste testfase met de Aruba CX kon ik via de logviewer van de TFTP-server volgen welke bestanden werden opgevraagd. Eerst werd het firmwarebestand gedetecteerd en het laadproces werd gestart, gevolgd door het configuratiebestand. Bij een latere controle bleek dat enkel het configuratiebestand correct was opgehaald, terwijl de firmware niet werd bijgewerkt.

Na enkele pogingen om het probleem te achterhalen, heb ik mijn mentor op de stageplaats om hulp gevraagd. Hij gaf de tip om de Cisco-switch, die tussen de server en de Aruba-switch stond, weg te halen. In plaats daarvan heb ik een rechtstreekse verbinding opgezet tussen de Windows Server en de Aruba CX-switch, door gebruik te maken van een extra netwerkadapter.

```
Connection received from 192.168.0.10 on port 41388 [19/02 10:52:30.593]
Read request for file <ArubaOS-CX_6400-6300_10_13_1070.swi>. Mode octet [19/02 10:52:30.593]
OACK: <tsize=985274319,blksize=1468,timeout=1,> [19/02 10:52:30.609]
Using local port 64866 [19/02 10:52:30.609]
TIMEOUT waiting for Ack block #27328 [19/02 10:53:11.116]
Connection received from 192.168.0.10 on port 38366 [19/02 10:54:11.686]
Read request for file <ArubaOS-CX_6400-6300_10_13_1070.swi>. Mode octet [19/02 10:54:11.686]
OACK: <tsize=985274319,blksize=1468,timeout=1,> [19/02 10:54:11.686]
Using local port 54303 [19/02 10:54:11.686]
Connection received from 192.168.0.10 on port 38366 [19/02 10:54:13.889]
Read request for file <ArubaOS-CX_6400-6300_10_13_1070.swi>. Mode octet [19/02 10:54:13.889]
OACK: <tsize=985274319,blksize=1468,timeout=1,> [19/02 10:54:13.889]
Using local port 54304 [19/02 10:54:13.889]
Connection received from 192.168.0.10 on port 38366 [19/02 10:54:15.889]
Read request for file <ArubaOS-CX_6400-6300_10_13_1070.swi>. Mode octet [19/02 10:54:15.889]
OACK: <tsize=985274319,blksize=1468,timeout=1,> [19/02 10:54:15.889]
Using local port 54305 [19/02 10:54:15.889]
TIMEOUT waiting for Ack block #0 [19/02 10:54:17.742]
Connection received from 192.168.0.10 on port 43360 [19/02 10:54:18.682]
Read request for file <ArubaOS-CX_6400-6300_10_13_1070.swi>. Mode octet [19/02 10:54:18.682]
OACK: <tsize=985274319,blksize=1468,timeout=1,> [19/02 10:54:18.682]
Using local port 54306 [19/02 10:54:18.682]
TIMEOUT waiting for Ack block #0 [19/02 10:54:19.954]
TIMEOUT waiting for Ack block #0 [19/02 10:54:21.929]
TIMEOUT waiting for Ack block #11751 [19/02 10:54:39.632]
```

Figuur 21: Screenshot Firmware update fail

Tijdens de nieuwe test wijzigde het IP-adres van de serverinterface naar het adres van de netwerkadapter. Dit had echter geen impact op de werking en bleek uiteindelijk geen probleem te vormen. Nu werden zowel het configuratiebestand als de firmware correct opgehaald en toegepast door de switch.



Figuur 22: Screenshot Firmware update zonder Cisco Switch

De vermoedelijke oorzaak van het probleem lag bij de beperking in pakketgrootte (MTU - Maximum Transmission Unit).

TFTP maakt gebruik van het UDP-protocol om bestanden, zoals firmware, over het netwerk te verzenden. Deze bestanden worden opgedeeld in datapakketten. UDP kan deze pakketten niet automatisch aanpassen aan de max grootte die een tussenliggende netwerkcomponent aankan. De Cisco switch is niet in staat om grotere pakketten correct te verwerken of door te sturen, dus wordt de verzending onderbroken. Hierdoor komt het firmwarebestand niet volledig of correct aan bij de Aruba-switch.

5.4.1 Autostacking

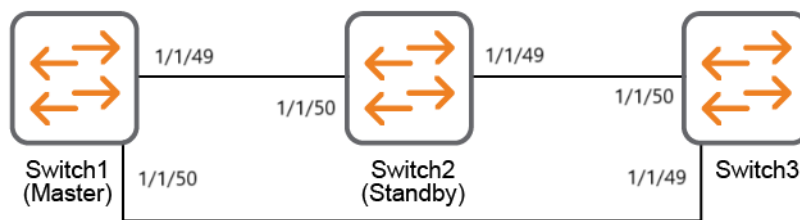
Nu de configuratiebestanden en firmware correct worden opgehaald kon ik het probleem rond autostacking aanpakken.

Vooraleer autostacking correct kan functioneren, moeten er vooraf enkele configuraties worden meegegeven aan de switches die deel zullen uitmaken van de stack. In een normale opstelling betekent dit dat elke switch afzonderlijk benaderd moet worden om manueel de stacking configuratie in te stellen.

Daarbij wordt onder andere vastgelegd welke member de switch zal worden binnen de stack en welke interfaces gebruikt zullen worden voor de onderlinge verbindingen tussen de switches. Daarna kunnen de DAC-kabels worden aangesloten om het stackingproces op te starten.

De onderstaande afbeelding tonen een voorbeeld van een stack bestaande uit drie switches, maar in totaal kunnen maximaal tien switches in één stack opgenomen worden. De verbinding tussen de switches verloopt telkens via interfacepoort 49 naar poort 50 van de volgende switch. De laatste switch in de keten verbindt uiteindelijk zijn poort 49 terug met poort 50 van de eerste switch, waardoor de stacking loop wordt gesloten.

Eenmaal de stacking is afgerond hebben enkel Switch 1 & 2 (de master en standby) de volledige configuratie.



Figuur 23: Foto voorbeeld Stacking interfaces (HPE Aruba Networking)

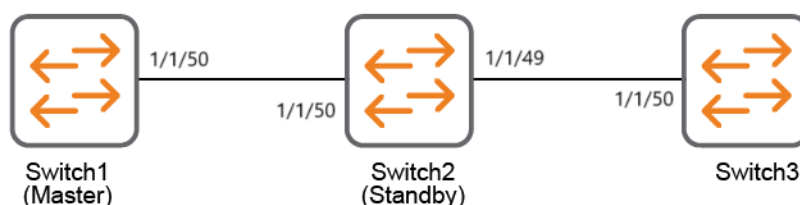
vsf member 1	vsf member 2	vsf member 3
type jl659a	type jl659a	type jl659a
link 1 1/1/49	link 1 2/1/49	link 1 3/1/49
link 2 1/1/50	link 2 2/1/50	link 2 3/1/50

Figuur 24: Voorbeeld VSF stacking interfaces

Dit bracht mij op het idee om de configuratie van de member-switches op te nemen in het basisconfiguratiebestand. Ik heb de volledige configuratie aangepast zodat de stackverbindingen op voorhand gedefinieerd zijn. Daarnaast heb ik ook de benodigde interfacepoorten toegevoegd in het configuratiebestand. Dit zorgt ervoor dat de bijkomende switches automatisch worden herkend als members van de stack. In mijn bestand heb ik standaard de configuratie voorzien voor een stack zes switches.

Ook deze test werd uitgevoerd en leverde in eerste instantie een probleem op. De test werd uitgevoerd met een stack van vier switches. Volgens de configuratie had switch 2 de rol van member 2 moeten krijgen, maar tijdens de initiële opstart kreeg hij de rol van member 4, terwijl switch 4 net de rol van member 2 (stand-by) kreeg.

Na grondig onderzoek, en met de hulp van mijn mentor, kwamen we tot de conclusie dat dit gedrag veroorzaakt werd door de manier waarop autostacking de primaire link interpreteert tussen de conductor en stand-by-switch. Voor een correcte automatische nummering is het essentieel dat switch 1 en switch 2 rechtstreeks met elkaar verbonden zijn via poort 50 (zoals weergegeven op de afbeelding hieronder). Daarnaast mag er nog geen retourverbinding zijn van de laatste switch naar de eerste tijdens de initiële opstart.



Figuur 25: aangepast voorbeeld interfaces VSF stacking voor Autostacking

Zodra de stack volledig is opgebouwd en de configuratie correct is toegepast, is het belangrijk om de fysieke bekabeling aan te passen. Dit betekent dat op switch 1 de verbinding met switch 2 verplaatst moet worden van poort 50 naar poort 49, zoals gespecificeerd in de running-configuratie.

De logische structuur van de stack blijft behouden, aangezien de rolverdeling tussen de switches (conductor, stand-by, members) op dat moment al is vastgelegd. Daarnaast kan ook de verbinding van de laatste member-switch naar switch 1 worden toegevoegd, zodat de ringtopologie volledig wordt gesloten.

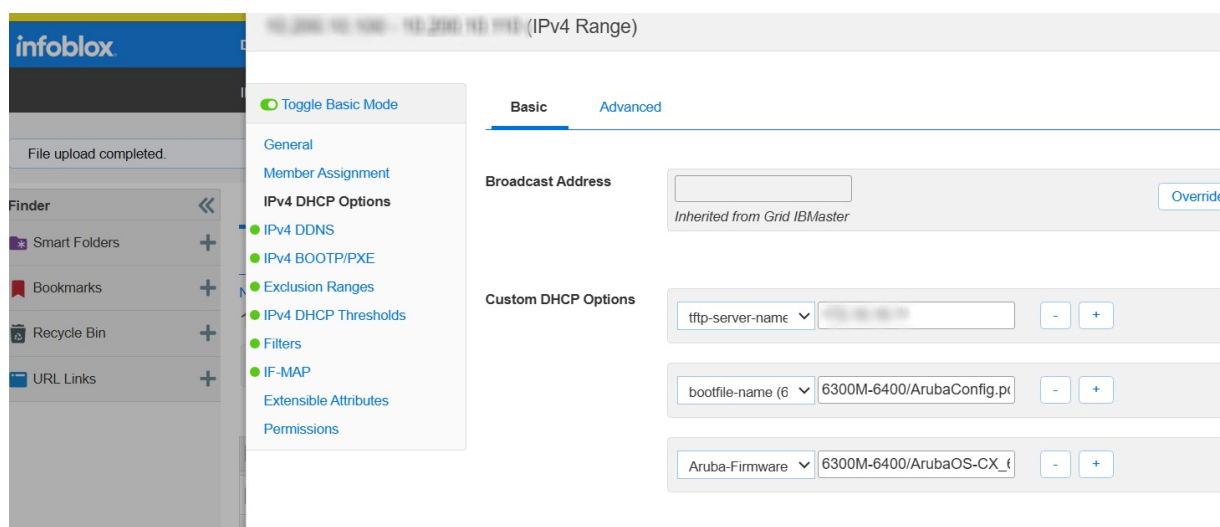
Het is aanbevolen om deze aanpassing pas uit te voeren wanneer de switches definitief gemonteerd zijn. Zo wordt er geen tijd verspild door het herstarten van de members.

Deze lokaal opgebouwde opstelling functioneert zoals bedoeld en is volledig mobiel. Ze kan eenvoudig meegenomen worden naar een externe locatie om ter plaatse meerdere switches gelijktijdig uit te rollen.

5.5 Infoblox

Nu mijn basisopstelling correct functioneerde in mijn lokale testomgeving, was het tijd om deze te integreren in het netwerk van het ziekenhuis. Hiervoor werd gebruikgemaakt van Infoblox als DHCP- en TFTP-server.

De configuratie die eerder op de Windows Server werd toegepast, werd overgenomen in Infoblox. Hierin werd een specifieke DHCP-range aangemaakt, waarin de nodige DHCP-sub opties werden toegevoegd. Deze sub-opties voorzien de switches van de juiste informatie om automatisch de configuratie en firmware op te halen



Figuur 26: Screenshot Infoblox voorbeeld

Via de deployment-switch kon het proces nu correct worden uitgevoerd. Door een specifieke VLAN aan te maken op zowel de core-switch als de deployment-switch, werd ervoor gezorgd dat het IP-adres van de Aruba CX switch bereikbaar is vanaf elke ICT-laptop binnen het netwerk.

5.5.1 Ondervonden problemen

Tijdens het toepassen van het Zero Touch Provisioning-proces met Infoblox, stootte ik op een probleem.

Bij het controleren van het resultaat bleek dat de configuratie wel correct werd toegevoegd, maar dat de firmware-upgrade niet was uitgevoerd. De switch draaide nog steeds op de oorspronkelijke firmwareversie.

Een mogelijke oorzaak hiervan is dat bepaalde firmwareversies niet rechtstreeks overschreven kunnen worden zonder eerst een tussenversie te installeren. In dat geval is de versieverschil te groot en wordt de update via ZTP geweigerd of overgeslagen. Dit verklaart waarom de configuratie correct werd geladen, maar de firmware niet werd bijgewerkt. Tijdens een eerdere test met mijn lokaal systeem heb ik dit probleem niet ondervonden, aangezien die test werd uitgevoerd met een al gebruikte CX-switch waarop al een recentere firmware geïnstalleerd was. Dit verschilt als enige met de nieuw geleverde switches waarmee ik aan het experimenteren was met Infoblox en autostacking. Deze toestellen draaien nog op een oudere firmware, wat de oorzaak kan zijn van het probleem met de automatische upgrade.

Na bijkomende tests en verder onderzoek bleek het probleem vermoedelijk bij Infoblox te liggen. Met de nieuwere switches lukt het wel om de firmware updates lokaal uit te voeren met mijn Windows Server omgeving. Daarbij speelt het geen rol hoe oud de huidige firmware is, of hoe recent de nieuwe versie de update verloopt telkens zonder problemen. Dit probleem kan ik niet oplossen op de Infoblox omgeving.

Omdat dit probleem zich enkel voordoet binnen de Infoblox-omgeving en mijn volledige ZTP-opstelling correct functioneert in de lokale testomgeving met Windows Server, heeft dit geen impact op mijn project. Samen met mijn mentor is er al reeds een alternatieve manier uitgewerkt om de firmware-updates manueel uit te voeren.

In andere netwerkomgevingen zal dit probleem zich dan ook niet voordoen.

6 Netmiko

Hoewel het proces succesvol is afgerond, blijft het een gemiste kans dat er nog steeds een reeks commando's handmatig moet worden ingevoerd. Dit strookt niet met mijn initieel plan om het ontwikkelen van een volledig geautomatiseerd Zero Touch Provisioning.

Daarom ben ik verder op zoek gegaan naar manieren om dit laatste stuk van het proces te automatiseren. Eén technologie die meteen mijn aandacht trok, is Netmiko. Tijdens het eerste jaar van mijn opleiding heb ik hiermee gewerkt, en het bleek een veelbelovende oplossing om resterende configuratiecommando's efficiënt en automatisch naar de switch te versturen.

Er waren natuurlijk verschillende andere mogelijkheden, maar gezien mijn eerdere ervaring met Python en het gebruik van Netmiko, was deze oplossing voor mij de meest voor logische keuze.

Voor de start van dit onderdeel had ik eerst wat herhaling van Python, gecombineerd met toch wat extra diepgang in de materie. Niet alleen omdat het al een tijd geleden was dat ik er nog mee gewerkt had, maar ook omdat de functionaliteiten die ik nu wilde realiseren verder gingen dan wat tijdens de lessen werd aangeleerd.

Wat begon als een eenvoudige opdracht het automatisch doorsturen van een handvol commando's op basis van wat gebruikersinput groeide al snel uit tot iets complexer dan ik voorzien had.

6.1 Wat doet het python script

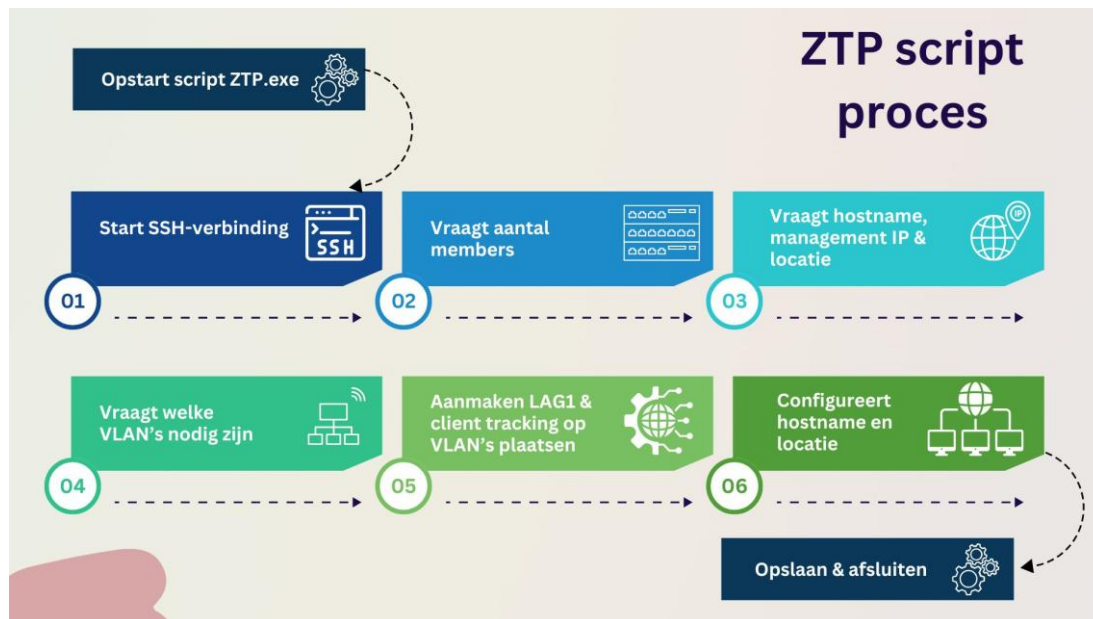
Het script maakt verbinding via SSH, de gebruiker voert handmatig het IP-adres van de switch in. De gebruikersnaam en het wachtwoord zijn hardcoded en maken deel uit van de standaardconfiguratie op de switch. Dit wachtwoord wordt enkel gebruikt tijdens het configuratieproces. Het kan niet worden gebruikt om aan te melden, behalve via een rechtstreekse consoleverbinding.

Bij het opstarten vraagt het script hoeveel stack members er zijn. Er zijn standaard zes stackleden aangemaakt, het script verwijdert automatisch op basis van de input. Dan wordt het management-IP-adres ingesteld binnen VLAN 2.

Daarna worden VLAN's toegevoegd aan de switch. Deze worden ingelezen uit een extern tekstbestand (Vlans.txt), dat door de gebruiker kan worden aangepast. Tijdens het lopen van het script wordt gevraagd welke van deze VLAN's moeten worden geconfigureerd. Er is ook een ingebouwde failsafe om zeker te zijn dat een blackhole VLAN wordt toegevoegd, en dit VLAN wordt ook gebruikt om alle fysieke interfaces aan toe te wijzen.

De interface LAG1 is al gedeeltelijk voorbereid in de basisconfiguratie, maar wordt met dit script afgewerkt door de juiste VLAN's toe te voegen als trunk-allowed VLAN's. Ook wordt client-track-ip geactiveerd op alle geselecteerde VLAN's, zodat de switch actief IP-adressen op deze VLAN's kan volgen.

Tenslotte vraagt het script de locatie, SNMP-instellingen en een hostname op via gebruikersinput, waarna deze worden toegepast. Door dit alles te combineren, automatiseert het script een groot deel van de basisconfiguratie van een nieuwe switch installatie.



Figuur 27 Schema werking python script

6.2 Uitleg script

6.2.1 Aanmelding

```
from netmiko import ConnectHandler

# Ip adres van de DHCP
host = input("Enter the DHCP-assigned IP of the switch: ")

# Aruba CX aanmeld gegevens
aruba_cx = {
    "device_type": "aruba_aoscx",
    "host": host,
    "username": " ",
    "password": " ",
}
```

Figuur 28: Screenshot Script aanmeld gegevens

1. Importeren van de Netmiko-module

De klasse ConnectHandler wordt geïmporteerd uit de Netmiko-bibliotheek. Deze is belangrijk om een SSH-verbinding te maken met de switch.

2. Invoeren van het IP-adres

Met de input() wordt aan de gebruiker gevraagd om het IP-adres in te geven dat door de DHCP-server is geleased aan de switch.

3. Opstellen van de verbindingsgegevens

In de aruba_cx dictionary zitten alle gegevens die nodig zijn om in te loggen op de switch deze informatie die wordt later gebruikt door Netmiko om verbinding te maken.

Elke lijn in de dictionary geeft iets specifiek aan:

- **device_type:** aruba_aoscx: geeft aan dat het script gebruikt wordt voor een Aruba CX-switch.
- **host:** gebruikt het ip adres ingevoerd door de user bij host input().
- **username & password:** Deze gegevens zijn al ingesteld via de basisconfiguratie en staan hardcoded in het script. Om veiligheidsredenen worden ze hier niet weergegeven.

6.2.2 Inlezen vlan.txt

```
# Leest VLAN-gegevens uit het aanpasbare bestand Vlans.txt.
def read_vlans_from_file(filename="Vlans.txt"):
    vlan_dict = {}
    try:
        with open(filename, "r") as file:
            for line in file:
                parts = line.strip().split(",")
                if len(parts) >= 2:
                    vlan_id, vlan_name = parts[:2]
                    vlan_dict[vlan_id.strip()] = vlan_name.strip()
    except FileNotFoundError:
        print(f"Error: {filename} not found. Please make sure the file exists in the same directory.")
    except Exception as e:
        print(f"Error reading VLANs from {filename}: {e}")
    return vlan_dict
```

Figuur 29: Screenshot Script inlezen vlan.txt

1. **def read_vlans_from_file(filename="Vlans.txt")**

Dit definieert een functie die een bestandsnaam verwacht, het bestand Vlans.txt

2. **Vlan_dict = {}**

Hier wordt een leeg woordenboek aangemaakt hier zullen later de VLAN-ID's en hun namen worden opgeslagen.

3. **Try het bestand openen en lezen**

Het bestand wordt geopend in leesmodus ("r")

- **Parts:** Elke regel in het bestand wordt opgekuist met strip() dit verwijdert spaties aan het begin en einde en wordt daarna gesplitst op de komma. Als er minstens twee elementen in de lijst zitten, worden deze geïnterpreteerd als VLAN-ID en VLAN-naam.
- **Except:** Dit registreert fouten, zoals wanneer het bestand niet gevonden wordt (FileNotFoundError) of wanneer er een andere fout optreedt tijdens het lezen van de inhoud.

4. **Return vlan_dict**

De functie wordt afgesloten en geeft het resultaat (de VLAN-gegevens) terug. Dit resultaat wordt vervolgens opgeslagen in de variabele vlan_dict, zodat het later in het script verder gebruikt kan worden.

6.2.3 VLAN-input van de gebruiker

```
# vraagt de gebruiker om op te geven welke VLANs geconfigureerd moeten worden
def get_vlans_from_user():
    vlan_list = []
    while True:
        vlan_input = input("Choose VLAN (enter 0 to finish): ").strip()
        if vlan_input == "0":
            break
        if vlan_input.isdigit():
            vlan_list.append(vlan_input)
        else:
            print("Invalid VLAN number. Please enter only digits.")
    return vlan_list
```

Figuur 30: Screenshot Script input van de user

1. Def get_vlans_from_user()

Dit is een functie die de gebruiker vraagt om handmatig VLAN-nummers in te voeren die vervolgens op de switch geconfigureerd zullen worden.

2. Vlan_list = []

Er wordt een lege lijst aangemaakt waarin de door de gebruiker ingegeven VLAN-nummers opgeslagen worden.

3. While True

Er wordt een oneindige lus opgestart zodat de gebruiker kan blijven VLAN-nummers invoeren. De lus blijft actief tot de gebruiker het getal 0 invoert.

- if vlan_input == input ("Choose vlan")

De vraag om VLAN-nummers in te voeren.

- .strip(): Verwijdert eventuele spaties in het begin en einde van de invoer.

- if vlan_input == "0":

Als de gebruiker 0 invoert dan wil dat zeggen dat hij klaar is met invoeren van VLAN's, de lus wordt dan afgebroken met het break commando.

- if vlan_input.isdigit():

Hier wordt gecontroleerd of de invoer enkel uit cijfers bestaat. als dit het geval is, wordt het VLAN nummer toegevoegd aan de lijst via vlan_list.append(vlan_input).

- else

Als de ingevoerde data geen cijfer is dan krijgt de gebruiker een foutmelding.

4. Retun vlan_list

Als de invoer is afgerond en de lus is gestopt, wordt de lijst met VLAN nummers teruggegeven. Deze lijst kan dangebruikt worden om de VLAN configuratie op de switch toe te voegen.

6.2.4 Registreren members & interfaces

```
# Genereert een interface range op basis van ingevoerde hoeveelheid stack members
def generate_interface_range(stack_size):
    interface_ranges = [f"{i}/1/1-{i}/1/48" for i in range(1, stack_size + 1)]
    return ",".join(interface_ranges)

try:
    print("\nConnecting to the Aruba CX switch...")
    net_connect = ConnectHandler(**aruba_cx)
    print("Connection successful!")

    while True:
        try:
            stack_size = int(input("Enter the number of required stack members (1-6): "))
            if 1 <= stack_size <= 6:
                break
            print(f"Invalid stack size entered: {stack_size}")
            print("Invalid input. Please enter a number between 1 and 6.")
        except ValueError:
            print("Invalid stack size input (non-integer).")
            print("Invalid input. Please enter a valid number.")

    interface_range = generate_interface_range(stack_size)
    hostname = input("Enter new hostname: ")
    mgmt_ip = input("Enter IP address for VLAN 3200: ")
    mgmt_mask = input("Enter subnet mask for VLAN 3200: ")
    Location = input("Location of the Switch : ")
    vlan_dict = read_vlans_from_file("Vlans.txt")
```

Figuur 31: Screenshot Script stacking & interfaces

1. Def generate_interface_range(stack_size)

De interfaces worden gegenereerd op basis van het aantal opgegeven stackleden. De variabele `i` wordt telkens gebruikt als verwijzing naar het nummer van een stack member. Voor elke waarde van `i` wordt een interface range opgebouwd van poort 1 tot en met poort 48 voor die specifieke switch.

- `for i in range(1, stack_size + 1)` zorgt ervoor dat de telling begint bij 1 en doorgaat tot en met het opgegeven aantal switches. De +1 is nodig omdat Python de laatste waarde in een `range()` standaard uitsluit.

2. ConnectHandler

Connecteer via SSH met de switch met gebruik van de `aruba_cx` dictionary.

3. Try Stack size

Hier wordt gevraagd aan de gebruiker hoeveel members er in de stack zitten. Er kan een keuze opgegeven worden van 1 tem 6. Als de invoer geldig is wordt de lus doorbroken, wordt er een ongeldig nummer gegeven dan volgt er een foutmelding samen met een nieuwe kans om het correct in te vullen.

4. Extra invoer

Hier wordt dan nog wat extra informatie gevraagd van de gebruiker om de switch te kunnen configureren.

- **Hostname:** De gebruiker wordt via een input veld gevraagd om een naam op te geven voor de switch.
- **Mgmt_ip:** Het IP-adres dat toegewezen wordt aan de management VLAN.
- **Mgmt_mask:** Het subnetmask dat hoort bij het opgegeven management-IP-adres.
- **Location:** is de fysieke locatie waar de switch komt te staan.
- **Vlan_dict:** Een woordenboek die alle VLAN's bevat en op dit punt in het script wordt ingelezen.

6.2.5 Verwijderen van overbodige members

```
# verwijderen van de extra members
print("\nRemoving unnecessary stack members...")

for i in range(6, stack_size, -1):
    print(f"removing VSF member {i}...")

    # stuur commando voor conf t
    net_connect.config_mode()

    # stuurt de command om Members te verwijderen + de input van de user
    output = net_connect.send_command_timing(f"no vsf member {i}")

    # controleert de welke output komt en vergelijkt
    if "Do you want to continue" in output:
        confirmation = net_connect.send_command_timing("y")

    net_connect.exit_config_mode()
```

Figuur 32: Screenshot Script verwijderen VSF members

1. **for i in range(6, stack_size, -1)**

Dit is een lus die achterwaarts telt van 6 naar het opgegeven aantal stackmembers. Wanneer de gebruiker bijvoorbeeld 3 members invoert, worden member 6, member 5 en member 4 verwijderd uit de configuratie.

2. **net_connect.config_mode()**

Dit is het commando dat wordt verstuurd om de switch in de configuratiemodus te plaatsen. De switch moet zich in deze hier bevinden voordat configuratiecommando's kunnen worden uitgevoerd.

3. **output = net_connect.send_command_timing(f"no vsf member {i}")**

Dit commando verwijdert een VSF stack member en slaat de uitvoer van dat commando op in een variabele.

4. **if "Do you want to continue" in output**

Hier wordt gecontroleerd of de switch om een bevestiging vraagt. Als dat het geval is, wordt automatisch het antwoord "Yes" verzonden.

5. **net_connect.exit_config_mode()** hiermee wordt er terug weggegaan uit de configuratie modus van de switch.

6.2.6 Configureren van de management interface

```
print("\nConfiguring Management VLAN (2)...")
net_connect.send_config_set([
    "vlan 2",
    "name Management",
    "interface vlan 2",
    f"ip address {mgmt_ip} {mgmt_mask}",
    "no shutdown",
    "exit",
    f"snmp-server system-location {Location}",
    f"snmp-server system-description {Location}",
    f"ip source-interface radius {mgmt_ip}",
])
```

Figuur 33 Screenshot Script configureren Mgmt Interface

1. net_connect.send_config_set([])

Met dit commando wordt een set configuratie-instructies verzonden naar de switch. De opdrachten die tussen de haakjes staan, worden uitgevoerd in de configuratiemodus.

De configuratieset bevat onder andere:

- **vlan 2:** VLAN 2 wordt aangemaakt, en wordt gebruikt als management VLAN. De naam van de VLAN wordt ook meegegeven.
- **interface vlan 2:** Opent de virtuele interface van VLAN 2.
- **IP adres:** Stelt het IP-adres en subnetmask in zoals ingevuld door de gebruiker via de input prompts.
- **no shutdown:** dit is toegevoegd om zeker te zijn dat de interface actief is.
- **exit:** Verlaat de interface van VLAN 2.
- **ip source-interface radius {mgmt_ip}:** dit is ook het management IP-Adres die gebruikt wordt voor RADIUS-verkeer.
- **snmp-server system-location {location}:** Stelt de fysieke locatie in van de switch, zoals ingevuld door de gebruiker.
- **snmp-server system-description {location}:** Gebruikt dezelfde gebruikers input als location.

De laatste twee SNMP-instellingen zijn optioneel, maar bieden extra informatie die handig is voor netwerkbeheerders bij het monitoren van switches via SNMP-tools.

6.2.7 VLAN aanmaken

```
print("\nEnter VLAN numbers to add (enter 0 to stop):")
selected_vlans = get_vlans_from_user()
blackhole_vlans = {"666", "1666", "2666", "3666"}
blackhole_vlan = None

if selected_vlans:
    print("\nAdding VLANs...")
    for vlan_id in selected_vlans:
        vlan_name = vlan_dict.get(vlan_id, f"VLAN{vlan_id}")
        if vlan_id in blackhole_vlans:
            blackhole_vlan = vlan_id
            print(f"Detected Blackhole VLAN: {blackhole_vlan}")
        net_connect.send_config_set([
            f"vlan {vlan_id}",
            f"name {vlan_name}"
        ])
```

Figuur 34: Screenshot Script aanmaken VLAN

1. selected vlans

Dit is een lijst met VLAN-nummers die de gebruiker heeft ingevoerd via de functie `get_vlans_from_user()`

2. blackhole_vlans

dit is een vaste verzameling van VLAN-nummers die gebruikt worden als blackhole VLAN's. Deze zijn om ongebruikte poorten op de switch af te schermen van het netwerk.

- De variabele `blackhole_vlan` wordt gebruikt om het VLAN-nummer op te slaan als de gebruiker er één van deze kiest. Zo weet het script welk VLAN als blackhole moet worden ingesteld.

3. If selected_vlans

Als de gebruiker één of meerdere VLANs heeft ingevuld, wordt elke VLAN-nummer doorzocht. Het script zoekt de naam op in `Vlans.txt`.

Tijdens dit wordt ook gecontroleerd of de gebruiker één van de blackhole-VLAN's heeft ingevuld. Als dat zo is, wordt dit opgeslagen. Is er geen blackhole-VLAN geselecteerd, wordt dit later expliciet nog gevraagd.

4. net_connect.send_config_set([])

Met dit worden de commando's naar de switch gestuurd. In dit geval verstuurd het de juiste ID en de bijhorende naam van de geselecteerde VLAN's.

6.2.8 Expliciet blackhole navragen

```
if not blackhole_vlan:
    while True:
        blackhole_vlan = input("Enter the Blackhole VLAN (666, 1666, 2666, or 3666): ").strip()
        if blackhole_vlan in blackhole_vlans:
            net_connect.send_config_set([
                f"vlan {blackhole_vlan}",
                "name Blackhole"
            ])
            break
        else:
            print("Invalid Blackhole VLAN. Please choose from 666, 1666, 2666, or 3666.")
```

Figuur 35: Screenshot Script expliciet blackhole navragen

Dit deel van het script sluit aan op het inlezen en aanmaken van VLAN's. Als de gebruiker tijdens het invoeren van de VLAN's geen blackhole-VLAN heeft ingevuld, wordt hij in dit deel verplicht om te doen. Dit gebeurt via een herhalende loop die blijft gaan tot een geldig blackhole-VLAN is ingevuld. Zodra een correcte VLAN is gekozen (666, 1666, 2666 of 3666), wordt het VLAN aangemaakt op de switch en krijgt het de naam "Blackhole". Bij een ongeldige invoer krijgt de gebruiker een foutmelding en wordt de vraag nog eens gesteld. Op deze manier wordt is het zeker dat er steeds een blackhole-VLAN is in de configuratie van de switch.

6.2.9 interfaces in de Blackhole Vlan

```
print(f"\nAssigning all interfaces to Blackhole VLAN {blackhole_vlan}...")
net_connect.send_config_set([
    f"interface {interface_range}",
    "no shutdown",
    f"vlan access {blackhole_vlan}",
    "no power-over-ethernet",
    "exit"
```

Figuur 36: Screenshot Script interfaces in blackhole VLAN

1. net_connect.send_config_set ([...])

Dit is nogmaals het commando die een list met meerdere configuratie commando's verstuurt naar de switch.

- **Interface {interface_range}**: Met dit commando worden alle interfaces in de volledige stack geselecteerd. Dit gebeurt met de eerder aangemaakte variabele interface_range, die werd gecreëerd met het aantal opgegeven stackleden.
- **No shutdown**: Activeert alle interfaces zodat ze klaar zijn voor netwerkverbindingen.
- **Vlan access {blackhole_vlan}**: Met het commando "vlan access" worden alle geselecteerde interfaces toegewezen aan een specifieke VLAN in dit geval de blackhole-VLAN.
- **No PoE**: Power-over-Ethernet is een techniek waarbij apparaten stroom krijgen via de netwerkkabel. Een voorbeeld hiervan is een Wi-Fi access point, dat dankzij PoE geen aparte stroomadapter of stopcontact nodig heeft. Standaard blijft PoE uitgeschakeld en wordt het pas aangezet als een apparaat PoE nodig heeft.
- **Exit**: Sluit de range van interfaces terug af.

Het groeperen van alle ongebruikte poorten in een blackhole-VLAN is een best practice in netwerkbeveiliging. Zo vermijd je dat iemand zomaar een toestel aansluit op een vrije poort, want deze poort zit in een "geïsoleerd" VLAN zonder netwerktoegang.

6.2.10 Link Aggregation Group

```
allowed_vlans = ",".join(selected_vlans + ["1", "2", blackhole_vlan])
print(f"\nConfiguring LAG1 to allow VLANs: {allowed_vlans}...")
net_connect.send_config_set([
    "interface lag 1",
    "no shutdown",
    "vlan trunk native 1",
    f"vlan trunk allowed {allowed_vlans}"
])
```

Figuur 37: Screenshot Script LAG

1. Een Link Aggregation Group (LAG)

Dit is het nemen van meerdere netwerkpoorten en ze combineren tot één logische verbinding. Hierdoor verhoogt de totale bandbreedte en heb je extra betrouwbaarheid als er een kabel uitvalt of beschadigd raakt. Dit wordt in de vooraf gemaakte configuratie bestand aangemaakt. Nu moet deze enkel nog geconfigureerd worden. Dit kon nog niet gebeuren omdat er nog geen VLAN's waren om mee te configureren.

2. Allowed_vlan

Dit maakt een commando met alle VLAN's die op de switch staan (alle VLAN's die door de gebruiker zijn meegegeven + de native-VLAN + de management-VLAN + de blackhole-VLAN, hij plaatst deze in de variabele `allowed_vlans`).

3. Net_connect.send_config_set([...])

Stuurt nogmaals een lijst van commando's naar de switch.

- **Interface lag 1:** Opent de virtuele interface van Lag 1.
- **No shutdown:** Om zeker te zijn dat de interface openstaat.
- **Vlan trunk native:** Hiermee wordt een native VLAN ingesteld, die gebruikt wordt als VLAN voor verkeer dat niet in een VLAN zit (untagged). Deze VLAN speelt doorgang voor niet-getagde frames, zodat alle andere VLAN's correct over de verbinding kunnen verlopen.
- **Vlan trunk allowed:** Hier worden alle VLAN's toegewezen aan de LAG, zodat verkeer van deze VLAN's over de link mag passeren. Het script genereert hiervoor van zichzelf het juiste commando met alle eerder aangemaakte VLAN's. Op deze manier wordt, zelfs nadat de VLAN's apart zijn aangemaakt, toch correct ingesteld dat ze via de LAG mogen communiceren.

6.2.11 Client track ip

```
print("\nAdding Client track IP to VLANs...")
net_connect.send_config_set([
    f"vlan {' '.join(map(str, selected_vlans))}",
    "client track ip",
    "exit"
])
```

Figuur 38: Screenshot Script client track ip

1. Client track IP

Is iets dat gebruikt wordt op switches om IP-adressen te koppelen aan fysieke poorten en MAC-adressen. Het helpt netwerkbeheerders om exact te weten welk toestel met welk IP op welke poort is aangesloten.

2. `net_connect.send_config_set([...])`

Stuurt nogmaals een lijst met commando's.

- **vlan{...}**: Deze stap verzamelt alle VLAN's die op de switch aanwezig zijn en zet ze om in één string. Deze string wordt gebruikt als commando ongeveer hetzelfde als met een interface-rang. In plaats van een range interfaces, wordt hier een range VLAN's aangeduid.
Op die volledige range wordt het commando client track ip gezet. Deze is nu op alle VLAN's hetzelfde. Omdat client track ip op VLAN moet geconfigureerd worden. Heb ik dit ook in het script laten uitvoeren in plaats van in de basis configuratie tijdens het opstarten.
- **Exit**: Sluit de range van VLAN's.

6.2.12 Laatste commando's

```
print("\nSetting hostname...")
net_connect.send_config_set(["configure terminal", f"hostname {hostname}", "exit"])

print("\nWriting meme for pietje \U0001F609...")
net_connect.send_command("write memory")

print("\nVerifying configuration...")
running_config = net_connect.send_command("show running-config")
print(running_config)

net_connect.disconnect()
print("\nConfiguration complete. Connection closed.")

except Exception as e:
    print(f"\nERROR: {e}")
```

Figuur 39: Screenshot Script laatste commando's

1. **net_connect.send_config_set([...])**

Door dit commando wordt een reeks configuratie-instructies naar de switch gestuurd. In dit geval wordt het gebruikt om de hostname van de switch in te configureren

2. **net_connect.send_command([...])**

Dit commando verstuurt één commando naar de switch, zonder dat je in configuratiemodus hoeft te zijn. Hier wordt het gebruikt voor write memory, waarmee de huidige configuratie opgeslagen wordt. Dit is belangrijk om verlies van instellingen te voorkomen bij bijvoorbeeld een stroomonderbreking. Het wordt ook gebruikt om het commando show running-config uit te doen. Dit toont de actieve configuratie, zodat er nog een laatste controle is op de configuratie.

3. **net_connect.disconnect()**

Met dit commando wordt de SSH-sessie met de switch netjes afgesloten nadat alle configuraties voltooid zijn.

4. **Except Exception**

Als afsluiter vangt dit commando eventuele fouten op die tijdens het uitvoeren van het script opkomen. Als er iets misgaat, wordt de fout weergegeven op het scherm zodat je weet waar het probleem zich voordoet.

Conclusie

Toen ik aan dit project begon, had ik slechts een minimaal beeld van wat er allemaal komt kijken bij het automatiseren van netwerkconfiguraties op switches. Ik had al wel gehoord van Zero Touch Provisioning maar wist niet precies hoe dit in de praktijk werd toegepast. Doordat ik dit project grondig heb onderzocht en uitgewerkt, heb ik op korte tijd veel bijgeleerd vooral op technisch vlak maar ook wat op het gebied van logisch denken. In deze graduaatsproef onderzocht ik hoe ZTP kan helpen om sneller en efficiënter nieuwe switches te configureren binnen een netwerk, in dit geval het netwerk van het AZ Sint-Jan ziekenhuis in Brugge.

Mijn doel was om het configuratieproces zoveel mogelijk te automatiseren zodat het netwerkteam minder tijd kwijt is aan repetitieve handelingen.

Tijdens dit project werkte ik met verschillende types van Aruba switches, eerst met de oudere ArubaOS 2930F en later met de nieuwere Aruba CX 6300M. Door een mini-netwerkomgeving op te zetten met daarin een combinatie van DHCP, TFTP en een zelfgeschreven Python-script op basis van Netmiko, slaagde ik erin om het proces grotendeels te automatiseren.

Het systeem dat ik uitwerkte zorgt ervoor dat een switch automatisch een IP-adres krijgt, de juiste firmware kan downloaden en een basisconfiguratie toepast met de vooraf ingestelde DHCP opties. Daarna kunnen aanvullende configuraties, VLAN's en poortinstellingen worden doorgegeven via het Python-script. Dit zorgt voor een uniforme, veilige en herhaalbare manier van werken.

Wat me tijdens het proces opviel en frustreerde, was dat je niet alle informatie terugvindt in de officiële documentatie van Aruba. Zeker wanneer ik bewust zonder Aruba-software mijn systeem wil opzetten hierdoor botste ik op heel wat problemen. Door mijn zoektermen wat aan te passen en mij niet alleen te focussen op "Zero Touch Provisioning", vond ik toch nog wat waardevolle informatie. Ik heb zeer veel via gebruikerservaringen op Reddit bijgeleerd. Die bredere denkwijze hielp me vooral bij het opzetten van de basisopstelling en het schrijven van het Python-script.

Hoewel ik zeker niet zou zeggen dat ik nu een expert ben in Python, heb ik door middel van trial-and-error een werkend script kunnen ontwikkelen. Het was niet altijd evident, maar door telkens stapsgewijs te testen en fouten te analyseren, vond ik uiteindelijk telkens een oplossing.

Het ZTP-proces dat ik heb uitgewerkt vermindert de tijd om een switch klaar te maken. Als een netwerkbeheerder vroeger een aantal uur bezig was met de volledige configuratie, kan dat nu met enkele klikken opgestart worden. Het volledige proces duurt nog steeds ongeveer een uur, maar de actieve aandacht van de beheerder is verminderd tot een kwartier. De resterende tijd kan intussen aan andere taken besteed worden, wat zorgt voor grote tijdswinst.

Afsluitend ben ik tevreden en trots dat ik een werkende, praktische oplossing heb uitgewerkt die niet alleen inzetbaar is binnen het ziekenhuis, maar mits enkele aanpassingen ook bruikbaar is in andere netwerkomgevingen met Aruba of zelfs Cisco-switches. Als ik terugkijk op het traject, zou ik liever onmiddellijk begonnen met de Aruba CX 6300M. Dat had me enkele uren en frustraties bespaard. Toch ben ik zeer tevreden met hoe het project uiteindelijk verlopen is.

Ik had in het begin mijn twijfels bij de keuze van dit onderwerp. Dit was een voorstel van mijn stage mentor. Maar naarmate ik mee informatie verzamelde, groeide mijn interesse in dit onderwerp. Ik ben blij dat ik deze kans heb gekregen en ik kijk positief terug op het hele proces.

Referentie lijst

Aruba. (sd). *HPE-Aruba*. Opgehaald van

<https://buy.hpe.com/be/en/networking/switches/fixed-port-l3-managed-ethernet-switches/networking-cx-switch-series/hpe-aruba-networking-cx-6300m-24%E2%80%91port-1gbe-4%E2%80%91port-sfp56-switch/p/jl664a>

Kaspersky. (sd). *what is an ip address*. Opgehaald van kaspersky:

<https://www.kaspersky.nl/resource-center/definitions/what-is-an-ip-address>

Melkote, K. (2021, 6 1). *Let's name it aruba*. Retrieved from community.hpe.com:

<https://community.hpe.com/t5/networking/let-s-name-it-aruba>

Mensink, M. (2022, Februari 24). *ssh*. Opgehaald van hypernode:

<https://www.hypernode.com/nl/blog/ssh-complete-uitleg/>

one.com. (sd). *TFTP*. Opgehaald van one: <https://www.one.com/en/hosting/what-is-tftp>

Providercheck. (2022, Juni 9). *wat is subnet*. Opgehaald van providercheck:

<https://www.providercheck.nl/kennisbank/bericht/wat-is-subnet>

Pynetlabs. (2024, April 24). *What is Netmiko*. Opgehaald van pynetlabs:

https://www.pynetlabs.com/what-is-netmiko-and-how-to-use-it-in-python/#What_is_Netmiko

Scholten, A. (2024, 3 8). *windows server 2022*. Opgehaald van softtrader:

<https://softtrader.nl/microsoft-software/windows-server/windows-server-2022/>

truegen. (sd). *infoblox*. Opgehaald van truegen: <https://www.truegen.be>

Wikipedia. (sd). *DHCP*. Opgehaald van wikipedia:

https://nl.wikipedia.org/wiki/Dynamic_Host_Configuration_Protocol

Workingtalent. (sd). *wat is python*. Opgehaald van workingtalent:

<https://www.workingtalent.nl/wat-is-python>

Ziekenhuis, S.-J. (sd). *AZ-Sintjan historiek*. Opgehaald van azsintjan:

www.azsintjan.be/historiek

Bijlagen

Handleiding Zero Touch Provisioning ArubaCX 6300M & Python Script

Dit is een QR-code die verwijst naar mijn persoonlijke GitHub-pagina. Op deze pagina zijn de bijlagen van zowel de handleiding als het script terug te vinden.

